



Department of Computer Science

Lab Manual

of

SPEECH PROCESSING AND RECOGNITION

MAI574

Class Name: V MScAIML

**Master of Science Artificial Intelligence and
Machine Learning**

2025-26

Prepared by: Soujanya Bhat

Verified by: Faculty name:

Department Overview

The Department of Computer Science of CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape the nation's destiny. The training imparted aims to prepare young minds for the challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field.

Vision

The Department of Computer Science endeavours to imbibe the vision of the University “**Excellence and Service**”. The department is committed to this philosophy which pervades every aspect and functioning of the department.

Mission

“To develop IT professionals with ethical and human values”. To accomplish our mission, the department encourages students to apply their acquired knowledge and skills towards professional achievements in their career. The department also molds the students to be socially responsible and ethically sound.

Introduction to the Programme

Machines are gaining more intelligence to perform human-like tasks. Artificial Intelligence has spanned across the world irrespective of domains. MSc (Artificial Intelligence and Machine Learning) will enable to capitalize this wide spectrum of opportunities to the candidates who aspire to master the skill sets with a research bent. The curriculum supports the students to obtain adequate knowledge in the theory of artificial intelligence with hands-on experience in relevant domains with tools and techniques to address the latest demands from the industry. Also, candidates gain exposure to research models and industry standard application development in specialized domains through guest lectures, seminars, industry offered electives, projects, internships, etc.

Programme Objective

- To acquire in-depth understanding of the theoretical concepts in Artificial Intelligence and Machine Learning
- To gain practical experience in programming tools for Data Engineering, Knowledge Representation, Artificial intelligence, Machine learning, Natural Language Processing and Computer Vision.
- To strengthen the research and development of intelligent applications skills through specialization based real time projects.
- To imbibe quality research and develop solutions to social issues.

Programme Outcomes:

PO1 : Conduct investigation and develop innovative solutions for real world problems in industry and research establishments related to Artificial Intelligence and Machine Learning

PO2 : Apply programming principles and practices for developing automation solutions to meet future business and society needs.

PO3 : Ability to use or develop the right tools to develop high end intelligent systems

PO4 : Adopt professional and ethical practices in Artificial Intelligence application development

PO5 : Understand the importance and the judicious use of technology for the sustainability of the environment.

MAI574– SPEECH PROCESSING AND RECOGNITION

Total Teaching Hours for Semester: 75 (3+4)

Max Marks:150

Credits: 5

Course Objectives

This course enables the learners to understand fundamentals of speech recognition, speech production and representation. It also enables the learners to impart knowledge on automatic speech recognition and pattern comparison techniques. This course helps the learners to develop automatic speech recognition models for different applications.

Course Outcomes

After successful completion of this course students will be able to

CO1: Understand the speech signals and represent the signal in time and frequency domain.

CO2: Analyze different signal processing and speech recognition methods.

CO3: Implement pattern comparison techniques and Hidden Markov Models (HMM)

CO4: Develop speech recognition system for real time problems.

Unit-1

Teaching Hours: 15

FUNDAMENTALS OF SPEECH RECOGNITION

Introduction- The Paradigm for Speech Recognition- Brief History of speech recognition research- The Speech Signal: The process of speech production and perception in human beings- the speech production system- representing speech in time and frequency domain- speech sounds and features.

Lab Programs:

1. Implement the task that takes in the audio as input and converts it to text.
2. Apply Fourier transform and calculate a frequency spectrum for a signal in the time domain.

Unit-2

Teaching Hours: 15

2.1 APPROACHES TO AUTOMATIC SPEECH RECOGNITION BY MACHINE:

The acoustic phonetic approach-The pattern recognition approach-The artificial intelligence approach.

2.2 SIGNAL PROCESSING AND ANALYSIS METHODS FOR SPEECH RECOGNITION:

Introduction- spectral analysis models- the bank of filters front end processor- linear predictive coding model for speech recognition- vector quantization.

Lab Programs:

3. Implement sampling and quantization techniques for the given speech signals.
4. Explore linear predictive coding model for speech recognition

Unit-3

Teaching Hours: 15

PATTERN COMPARISON TECHNIQUES

Speech detection- distortion measure- Mathematical consideration- Distortion measure – Perceptual consideration- Spectral Distortion Measure- Incorporation of spectral dynamic feature into distortion measure- Time alignment and normalization.

Lab Programs:

0. Demonstrate different pattern in the given speech signal
0. Implement time alignment and normalization techniques

Unit-4

Teaching hours: 15

THEORY AND IMPLEMENTATION OF HIDDEN MARKOV MODELS:

Introduction- Discrete time Markov processes- Extension to hidden Markov Models- Coin - toss models- The urn and ball model- Elements of a Hidden Markov Model- HMM generator of observation- The three basic problems for HMM's- The Viterbi algorithm- Implementation issues for HMM's.

Lab Programs:

7. Implement simple hidden Markov Model for a particular application.
8. Apply Viterbi dynamic programming algorithm to find the most likely sequence of hidden states.

Unit-5

Teaching Hours: 15

TASK ORIENTED APPLICATION OF AUTOMATIC SPEECH RECOGNITION:

Task specific voice control and dialog- Characteristics of speech recognition applications- Methods of handling recognition error- Broad classes of speech recognition applications- Command and control applications- Voice repertory dialer- Automated call-type recognition- Call distribution by voice commands- Directory listing retrieval- Credit card sales validation.

Lab Programs:

9. Demonstrate automatic speech recognition for Call distribution by voice commands.
10. Apply speech recognition system to access telephone directory information from spoken spelled names (Directory listing retrieval).

Text Books and Reference Books

- [1] Fundamentals of Speech Recognition, Lawrence R Rabiner and Biing- Hwang Juang. Prentice-Hall Publications, 20209.
- [2] Introduction to Digital Speech Processing, Lawrence R. Rabiner, Ronald W. Schafer, Now Publishers, 2015.

Essential Reading / Recommended Reading

- [1] Intelligent Speech Signal Processing, Nilanjan Dey, Academic Press, 2019.
- [2] Speech Recognition-The Ultimate Step-By-Step Guide, Gerardus Blokdyk, 5STARCooks,2021.
- [3] Automatic Speech Recognition- A Deep Learning Approach, Dong Yu, Li Deng, Springer-Verlag London, 2015.

Web Resources:

1. www.w3cschools.com
2. <https://www.simplilearn.com/tutorials/python-tutorial/speech-recognition-in-python>
3. <https://realpython.com/python-speech-recognition/>
4. <https://cloud.google.com/speech-to-text/docs/tutorials>
5. <https://www.coursera.org/courses?query=speech%20recognition>
6. <https://pylessons.com/speech-recognition>

CO – PO Mapping

	PO1	PO2	PO3	PO4	PO5
CO1	3	1			1
CO2	2	2			1
CO3	1	3	1		2
CO4	1	1	3	1	3

LIST OF PROGRAMS

MSAIM 2024-2026

Sl.no	Title of lab Experiment	Page number	RBT	CO
1	<u>Sampling and Reconstruction of Speech Signals</u>	08	L3	CO1, 3
2	<u>Fourier Transform and Frequency Spectrum Analysis of Signals</u>	25	L3	CO2, 3
3	<u>Speech-to-Text Application for Accessibility</u>	43	L3	CO3
4	<u>Linear Predictive Coding (LPC) Model for Speech Recognition</u>	56	L3	CO3
5	<u>Time Alignment and Normalization</u>	64	L3	CO3
6	<u>Dynamic Time Warping (DTW)</u>	68	L3	CO3, 4
7	<u>Dynamic Time Warping Algorithm (for real time)</u>	74	L4	CO3
8	<u>Hidden Markov model (HMM)</u>	82	L3	CO3
9			L3	CO4
10			L5	CO4

Evaluation Rubrics:

- (1) Implementation: 5 marks.
- (2) Complexity and Validation: 3 marks.
- (3) Documentation & Writing the inference: 2 marks.

Submission Guidelines:

- Make a copy of the lab manual template with your <name_reg:no_subject name >.

- Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as lab number.
- Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
- Create a Git Repository in your profile <SPR lab-reg no> . Follow a different branch for each lab <Lab 1, Lab 2...>, and push the code to Git. The link should be provided in Google Classroom along with the PDF of the lab manual.
- Upload the PDF to Google Classroom before the deadline.

Lab 1

Lab Exercise I: Sampling and Reconstruction of Speech Signals

Aim

To study sampling and reconstruction of speech signals at different sampling rates, evaluate reconstruction using zero-order hold and linear interpolation, and implement the source-filter model to analyze the effect of filtering, sampling, and reconstruction on speech quality.

(1) Implement sampling and quantization techniques for the given speech signals.

- Plot the time domain representation of the original speech signal.
- Sample the speech signal at different sampling rates (e.g., 8kHz, 16kHz, and 44.1kHz).
- Plot sampled speech signal for each of these sampling rates.
- Using the sampled signals from above, reconstruct the signal using:
 - Zero-order hold (nearest-neighbor interpolation)
 - Linear interpolation.
- Calculate the Mean Squared Error (MSE) between the original and the reconstructed signals for both methods.

Write an inference on how sampling rates affect the quality and accuracy of the reconstructed speech signal.

(2) Implement the source-filter model for a given speech signal and analyze the impact of sampling and reconstruction on the quality of the speech signal.

(a) Generate a synthetic speech signal using the source-filter model.

(i) Create a source signal (e.g., a glottal pulse train for voiced sounds or white noise for unvoiced sounds).

(ii) Apply a filter that models the vocal tract, represented by an all-pole filter or an FIR filter with formants (resonances of the vocal tract).

(b) Plot the generated speech signal and analyze the effect of the filter on the original source.

(c) Sample the speech signal generated above at different sampling rates (e.g., 8 kHz, 16 kHz, 44.1 kHz).

(d) Reconstruct the signal using a suitable interpolation method (e.g., zero-order hold, linear interpolation).

(e) Compute the Mean Squared Error (MSE) between the original and reconstructed speech signals.

Write an inference on tasks such as creating the source-filter model, different sampling rates, and reconstruction of the sampled signals.

Code with Results

1. Sampling and Reconstruction of Speech

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.io import wavfile

# --- Load Audio ---

fs_original, speech = wavfile.read('/Users/soujanyabhat/Downloads/sample-9s.wav')

# If stereo, take first channel

if len(speech.shape) > 1:
```

```
speech = speech[:, 0]

# Normalize
speech = speech / np.max(np.abs(speech))

# Original time axis
t_original = np.arange(len(speech)) / fs_original

# Plot original
plt.figure(figsize=(12,4))
plt.plot(t_original, speech)
plt.title(f'Original Speech Signal (fs = {fs_original} Hz)')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.show()

# --- Downsample to different rates ---
sampling_rates = [8000, 16000, 22050, 44100, 48000]

sampled_signals = {}

for fs in sampling_rates:
    t_new = np.arange(0, len(speech)/fs_original, 1/fs)
    sampled_signal = np.interp(t_new*fs_original, np.arange(len(speech)), speech)
    sampled_signals[fs] = sampled_signal

# Plot sampled signal
plt.figure(figsize=(12,4))
```

```
plt.stem(t_new[:200], sampled_signal[:200], linefmt='r', markerfmt='ro', basefmt='k')

plt.title(f'Sampled Speech at {fs} Hz (first 200 samples)')

plt.xlabel('Time [s]')

plt.ylabel('Amplitude')

plt.show()


# --- Reconstruction Methods ---

def zero_order_hold(sampled, original_len):

    """True ZOH: hold each sample constant until next one."""

    x_old = np.arange(len(sampled))

    x_new = np.linspace(0, len(sampled)-1, original_len)

    zoh_signal = sampled[np.floor(x_new).astype(int)]

    return zoh_signal


def linear_interpolation(sampled, original_len):

    """Standard linear interpolation reconstruction."""

    x_old = np.arange(len(sampled))

    x_new = np.linspace(0, len(sampled)-1, original_len)

    lin_signal = np.interp(x_new, x_old, sampled)

    return lin_signal


# --- Reconstruction + MSE ---

mse_results = {}


for fs in sampling_rates:

    sampled = sampled_signals[fs]

    zoh = zero_order_hold(sampled, len(speech))
```

```
lin = linear_interpolation(sampled, len(speech))

mse_zoh = np.mean((speech - zoh)**2)

mse_lin = np.mean((speech - lin)**2)

mse_results[fs] = {'ZOH': mse_zoh, 'Linear': mse_lin}

# Plot reconstruction zoomed into small window

plt.figure(figsize=(12,4))

plt.plot(t_original[:500], speech[:500], label='Original')

plt.plot(t_original[:500], zoh[:500], label='ZOH Reconstruction')

plt.plot(t_original[:500], lin[:500], label='Linear Reconstruction')

plt.title(f'Reconstruction at {fs} Hz (zoomed first 500 samples)')

plt.xlabel('Time [s]')

plt.ylabel('Amplitude')

plt.legend()

plt.show()

print("MSE Results for different sampling rates:")

for fs in sampling_rates:

    print(f'{fs} Hz: ZOH = {mse_results[fs]["ZOH"]:.6f}, Linear = {mse_results[fs]["Linear"]:.6f}')

# --- Frequency Spectrum Comparison ---

def plot_spectrum(signal, fs, label):

    N = len(signal)

    freqs = np.fft.rfftfreq(N, 1/fs)

    spectrum = np.abs(np.fft.rfft(signal)) / N

    plt.plot(freqs, 20*np.log10(spectrum + 1e-12), label=label)
```

```
plt.figure(figsize=(12,6))

plot_spectrum(speech, fs_original, fOriginal ({fs_original} Hz'))

for fs in sampling_rates:

    plot_spectrum(sampled_signals[fs], fs, f'{fs} Hz sampled')

plt.title("Frequency Spectrum Comparison")

plt.xlabel("Frequency [Hz]")

plt.ylabel("Magnitude [dB]")

plt.legend()

plt.show()
```

MSE Results for different sampling rates:

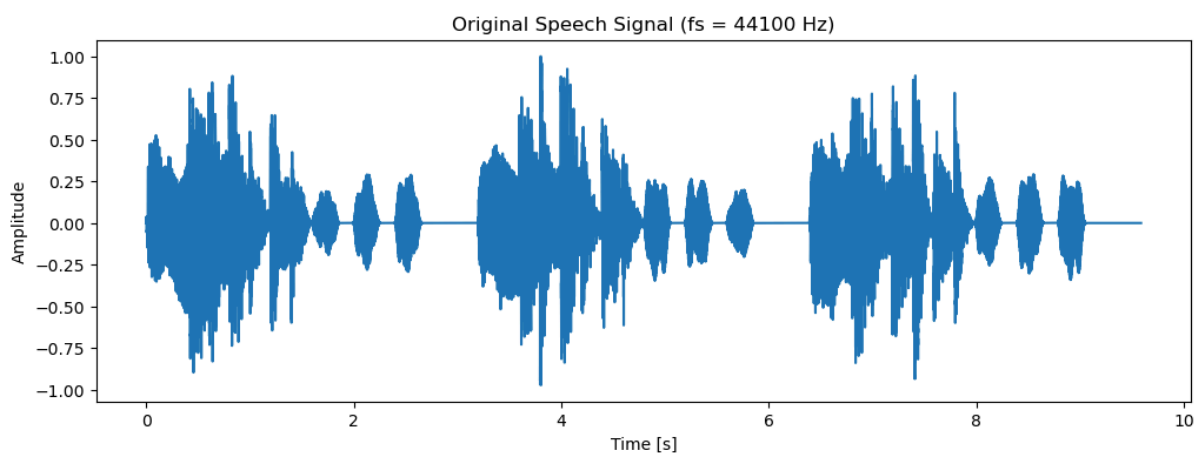
8000 Hz: ZOH = 0.001047, Linear = 0.000153

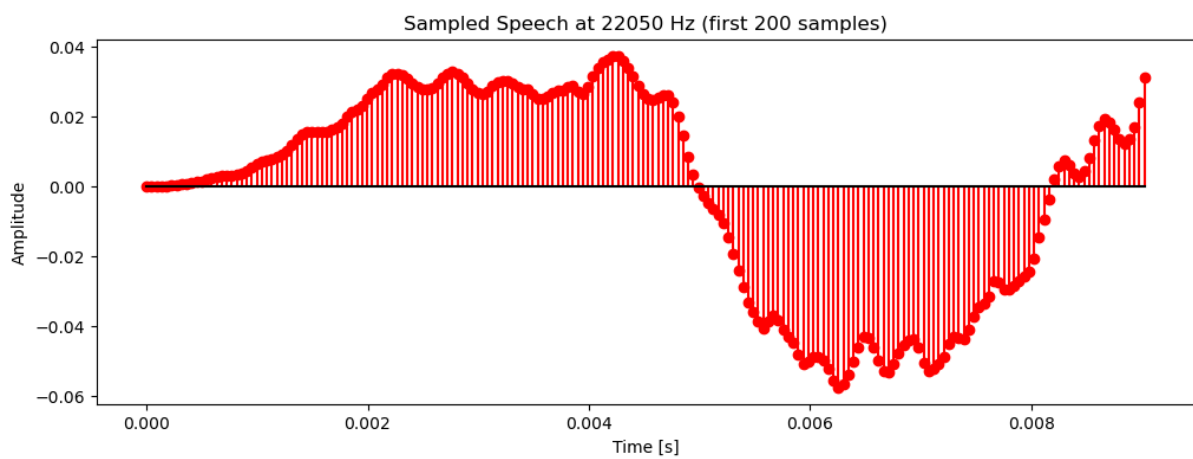
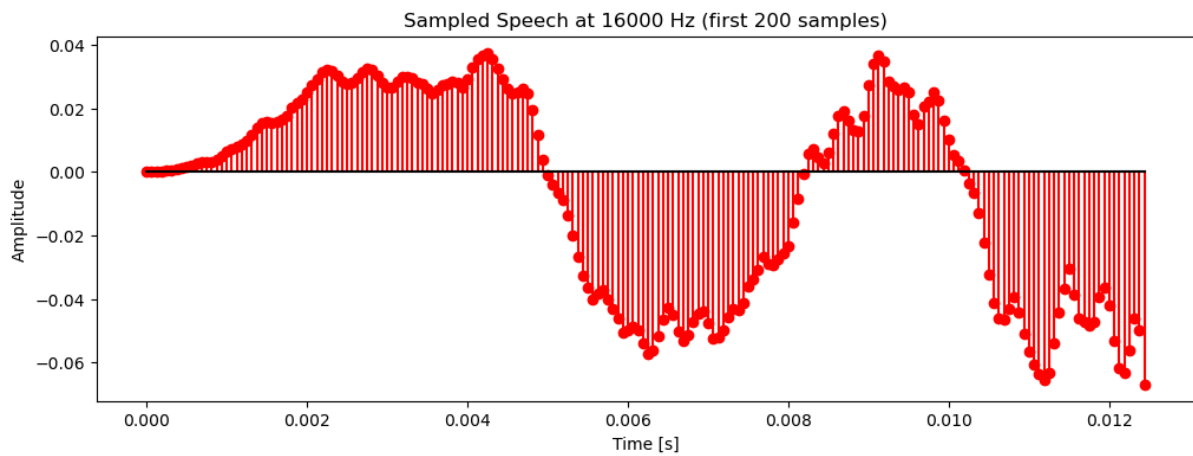
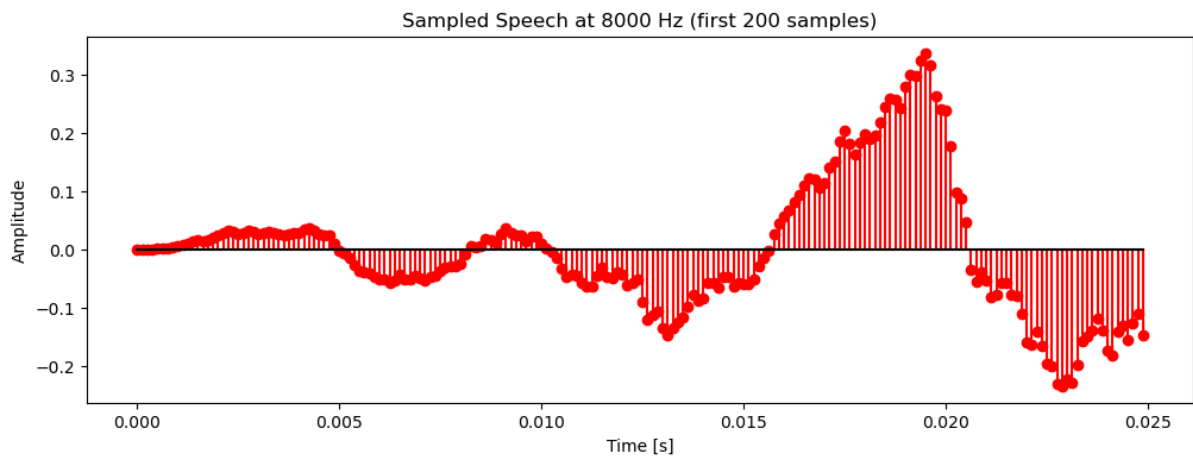
16000 Hz: ZOH = 0.000156, Linear = 0.000040

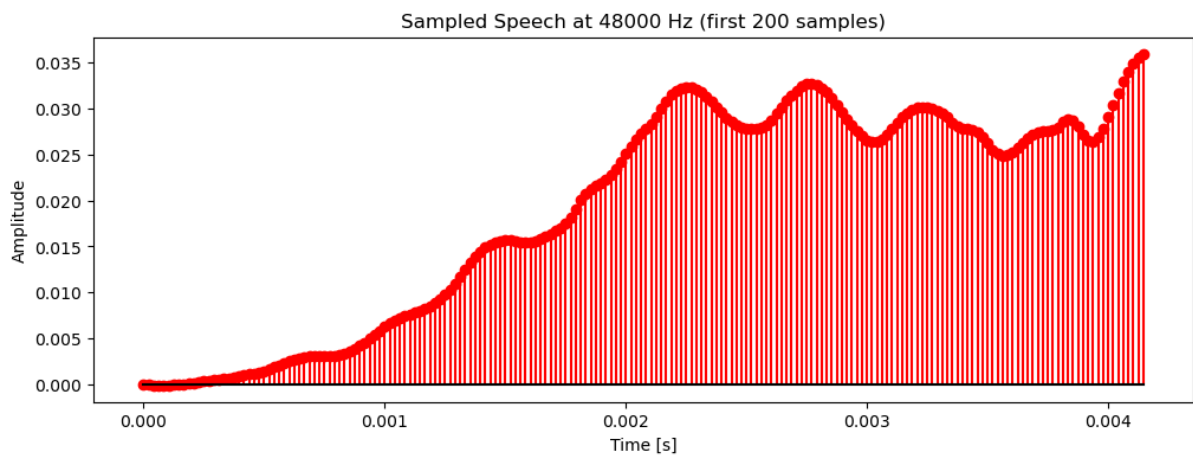
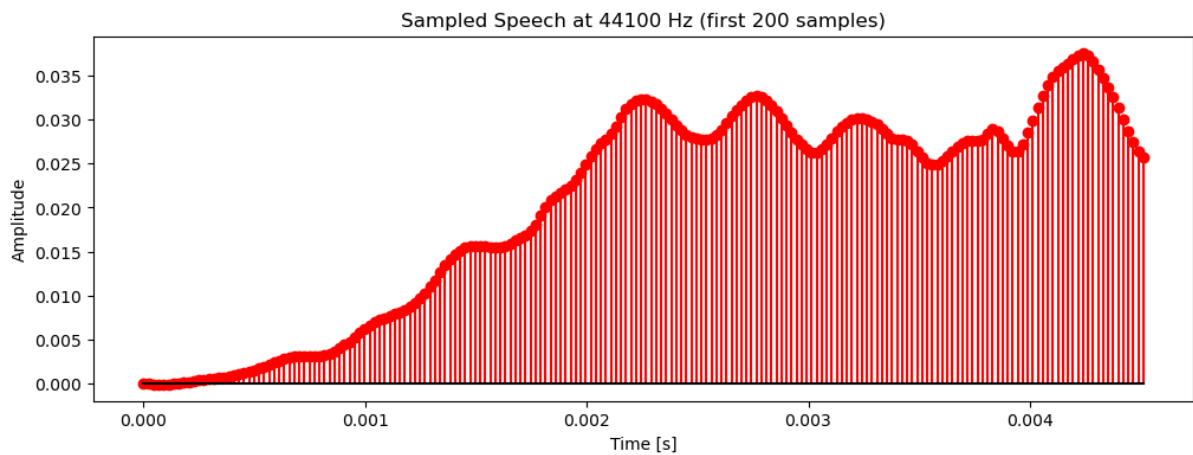
22050 Hz: ZOH = 0.000249, Linear = 0.000034

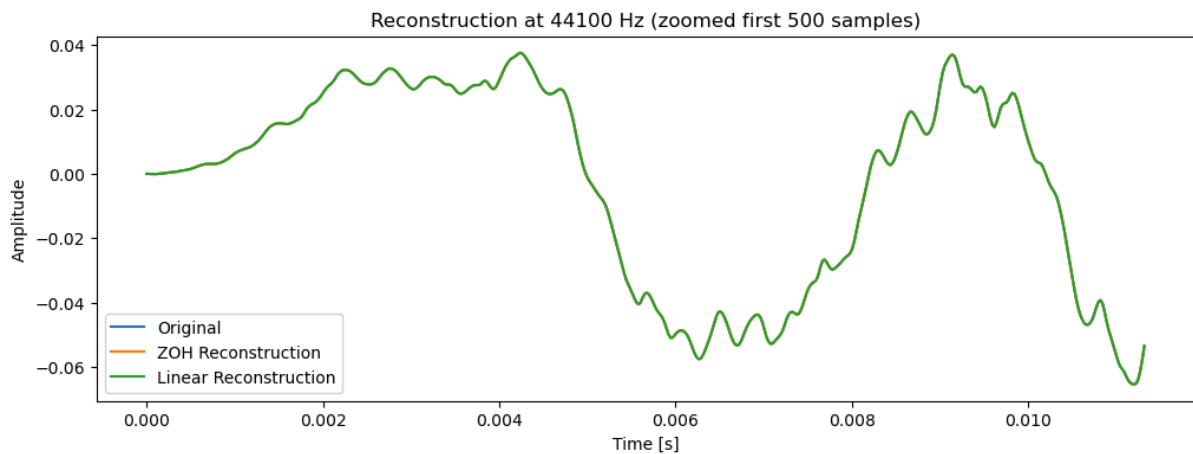
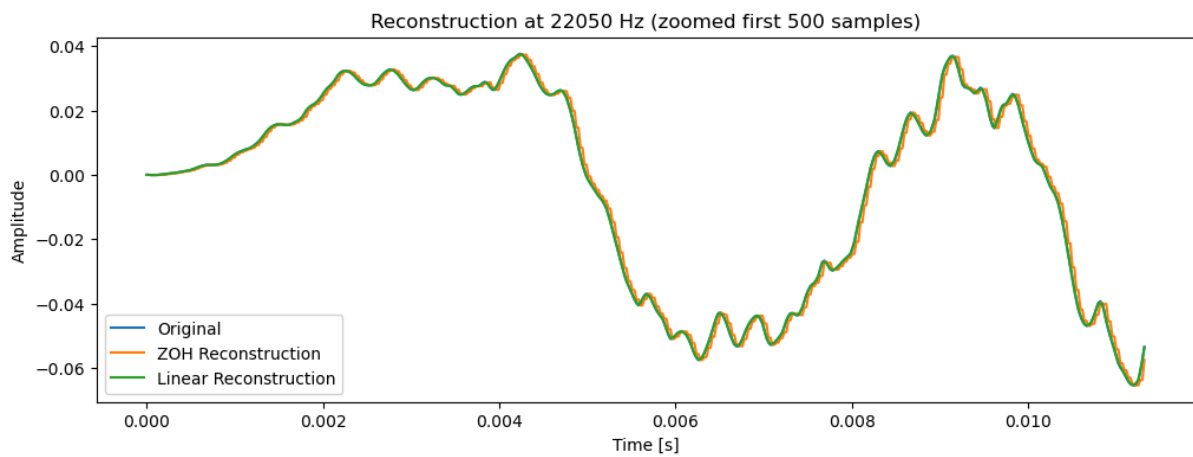
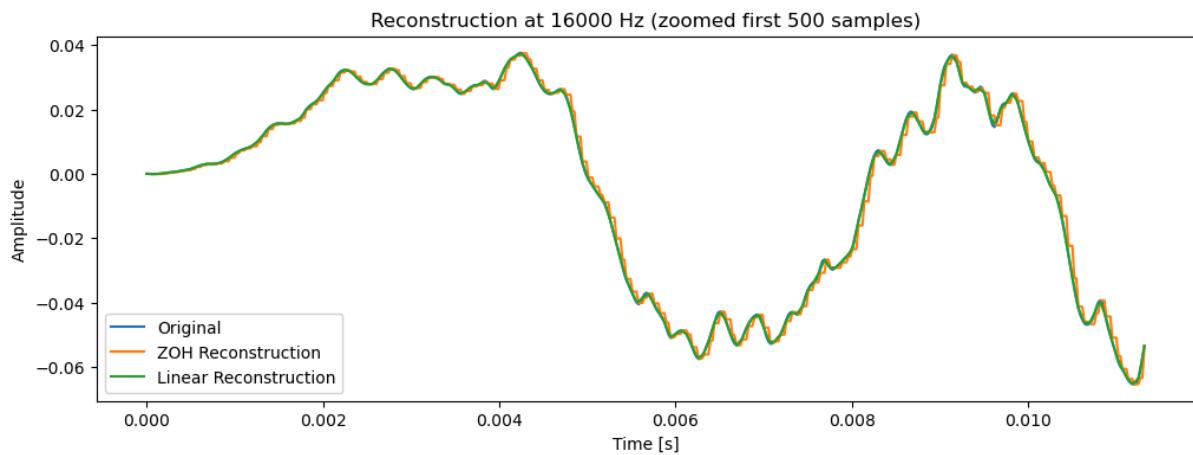
44100 Hz: ZOH = 0.000000, Linear = 0.000000

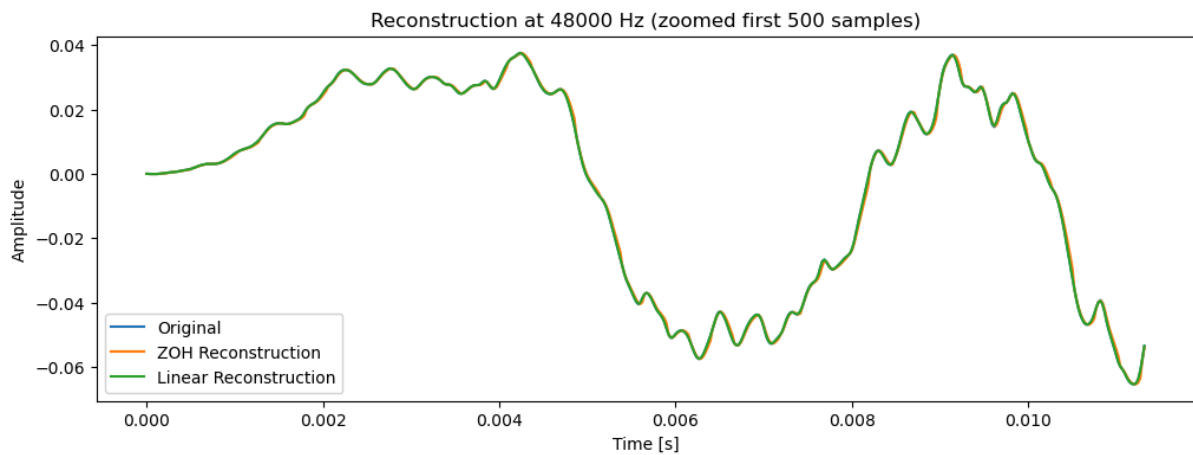
48000 Hz: ZOH = 0.000016, Linear = 0.000028











2. Source-Filter Model

```
import numpy as np

import matplotlib.pyplot as plt

from scipy import signal

from scipy.io import wavfile

# Synthetic source-filter signal

duration = 0.5 # seconds

fs = 44100

t = np.arange(0, duration, 1/fs)

f0 = 100 # fundamental

# make a richer source: sum of harmonics (decaying amplitude)

num_harmonics = 20

source_signal = np.zeros_like(t)

for k in range(1, num_harmonics+1):
```

```
source_signal += (1.0 / k) * np.sin(2*np.pi*(k*f0)*t)

# 2-formant resonators using iirpeak (narrower Q)

formants = [500, 1500]

bandwidths = [100, 200]

filtered_signal = np.zeros_like(source_signal)

for f1, bw in zip(formants, bandwidths):

    # Design bandpass-like resonator using iirpeak (approx)

    b, a = signal.iirpeak(f1 / (fs/2), Q=f1/bw) # center freq normalized to Nyquist

    filtered_signal += signal.lfilter(b, a, source_signal)

# normalize

filtered_signal /= np.max(np.abs(filtered_signal))

# show original waveform (zoomed)

plt.figure(figsize=(12,3))

plt.plot(t[:2000], filtered_signal[:2000])

plt.title("Synthetic Speech (source-filter) — zoom first 2000 samples")

plt.xlabel("Time [s]")

plt.ylabel("Amplitude")

plt.show()

def zero_order_hold(sampled, original_len):

    """True ZOH reconstruction from sampled array to original_len points"""

    x_old = np.arange(len(sampled))

    x_new = np.linspace(0, len(sampled)-1, original_len)

    zoh_signal = sampled[np.floor(x_new).astype(int)]
```

```
    return zoh_signal

def linear_interpolation(sampled, original_len):
    x_old = np.arange(len(sampled))
    x_new = np.linspace(0, len(sampled)-1, original_len)
    lin_signal = np.interp(x_new, x_old, sampled)
    return lin_signal

def plot_fft_db(sig, fs_local, label, Nfft=65536):
    # zero-pad/FFT for good resolution
    S = np.fft.rfft(sig, n=Nfft)
    freqs = np.fft.rfftfreq(Nfft, 1/fs_local)
    mag_db = 20*np.log10(np.abs(S) + 1e-12)
    plt.plot(freqs, mag_db, label=label)

def plot_spectrogram(sig, fs_local, title):
    f, tt, Sxx = signal.spectrogram(sig, fs_local, nperseg=1024, noverlap=768)
    plt.pcolormesh(tt, f, 20*np.log10(Sxx+1e-12), shading='gouraud')
    plt.ylabel('Frequency [Hz]')
    plt.xlabel('Time [s]')
    plt.title(title)
    plt.ylim(0, fs_local/2)
    plt.colorbar(label='Magnitude (dB)')

# ----- sampling rates to compare -----
sampling_rates = [8000, 16000, 22050, 44100, 48000]
```

```
# store signals

sampled_signals_proper = {}

sampled_signals_naive = {}

for fs_s in sampling_rates:

    # number of samples at new fs

    num_samples = int(len(filtered_signal) * fs_s / fs)

    # Proper resampling (uses FFT-based resample with anti-alias filtering)

    sampled_proper = signal.resample(filtered_signal, num_samples)

    sampled_signals_proper[fs_s] = sampled_proper


    # Naive sampling via linear interpolation (no anti-aliasing)

    t_new = np.arange(0, len(filtered_signal)/fs, 1/fs_s)

    sampled_naive = np.interp(t_new * fs, np.arange(len(filtered_signal)), filtered_signal)

    sampled_signals_naive[fs_s] = sampled_naive


    # plot a small region of sampled points to compare naive vs proper

    plt.figure(figsize=(10,2.5))

    # time axes for the zoomed region (~first 50 ms)

    zoom_len = int(0.05 * fs) # first 50 ms of original

    t_zoom = np.arange(zoom_len) / fs

    # reconstruct naive & proper back to original length via linear interp for plotting convenience

    recon_naive = linear_interpolation(sampled_naive, len(filtered_signal))

    recon_proper = linear_interpolation(sampled_proper, len(filtered_signal))

    plt.plot(t_zoom, filtered_signal[:zoom_len], label='Original (zoom)')

    plt.plot(t_zoom, recon_proper[:zoom_len], '--', label=f'Proper down-up {fs_s} Hz')

    plt.plot(t_zoom, recon_naive[:zoom_len], '.', label=f'Naive down-up {fs_s} Hz')
```

```
plt.title(f'Zoomed Time Comparison (first 50 ms) — target {fs_s} Hz')

plt.xlabel('Time [s]')

plt.legend()

plt.show()

# ----- Reconstruction and MSE (use proper downsampled signals) -----

mse_results_sfm = {}

for fs_s in sampling_rates:

    sampled = sampled_signals_proper[fs_s]

    # reconstruct to original length

    recon_zoh = zero_order_hold(sampled, len(filtered_signal))

    recon_lin = linear_interpolation(sampled, len(filtered_signal))

    mse_zoh = np.mean((filtered_signal - recon_zoh)**2)

    mse_lin = np.mean((filtered_signal - recon_lin)**2)

    mse_results_sfm[fs_s] = {'ZOH': mse_zoh, 'Linear': mse_lin}

print("MSE Results (proper resample -> recon to original length):")

for fs_s in sampling_rates:

    print(f'{fs_s} Hz: ZOH = {mse_results_sfm[fs_s]["ZOH"]:.8f}, Linear = {mse_results_sfm[fs_s]["Linear"]:.8f}')

# ----- FFT comparison -----

plt.figure(figsize=(12,5))

plot_fft_db(filtered_signal, fs, 'Original (44.1k)')

for fs_s in sampling_rates:

    # plot FFT of the downsampled signals with their own fs so Nyquist changes are visible

    s_down = sampled_signals_proper[fs_s]

    plot_fft_db(s_down, fs_s, f'Downsampled {fs_s} Hz')
```

```
plt.xlim(0, 12000) #focus to 12 kHz region (speech-relevant)

plt.ylim(-140, 20)

plt.xlabel('Frequency [Hz]')

plt.ylabel('Magnitude (dB)')

plt.title('FFT magnitude (dB) — original vs downsampled')

plt.legend()

plt.show()
```

MSE Results (proper resample -> recon to original length):

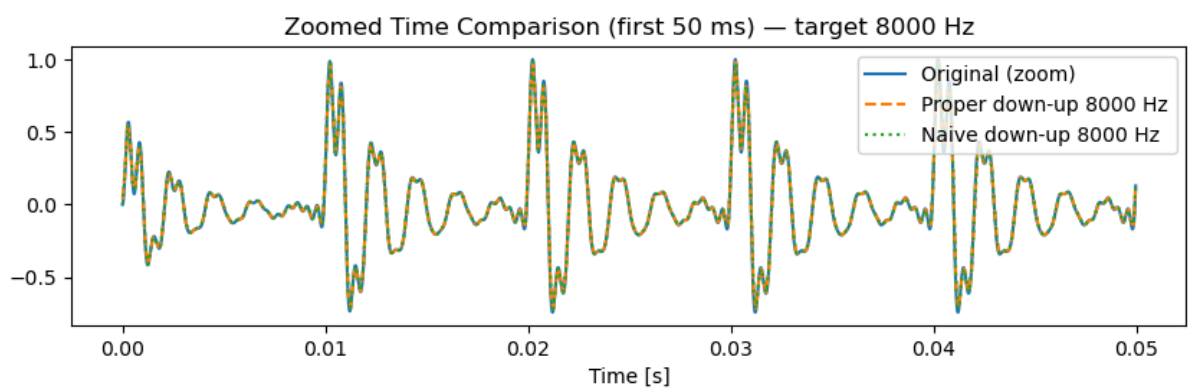
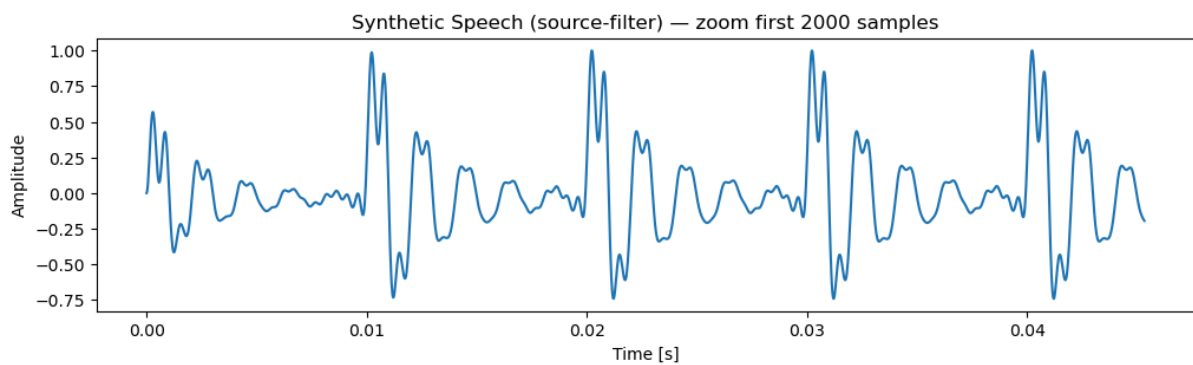
8000 Hz: ZOH = 0.02714471, Linear = 0.00654404

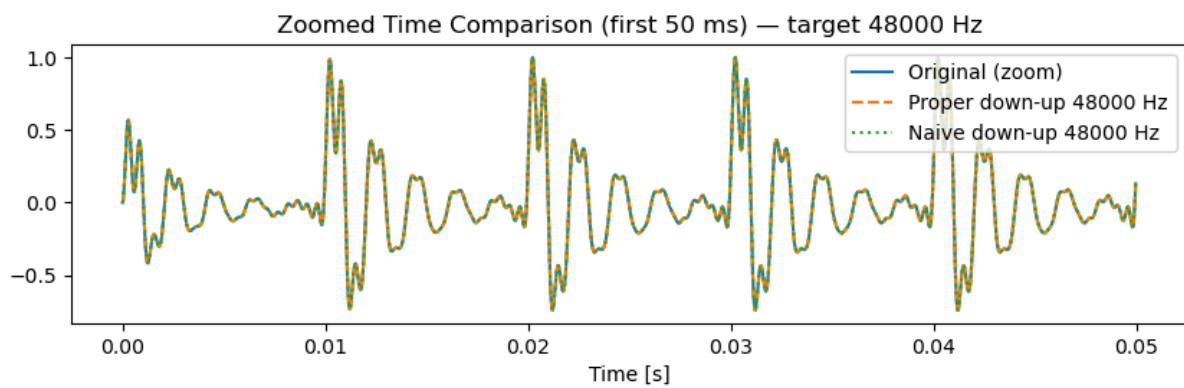
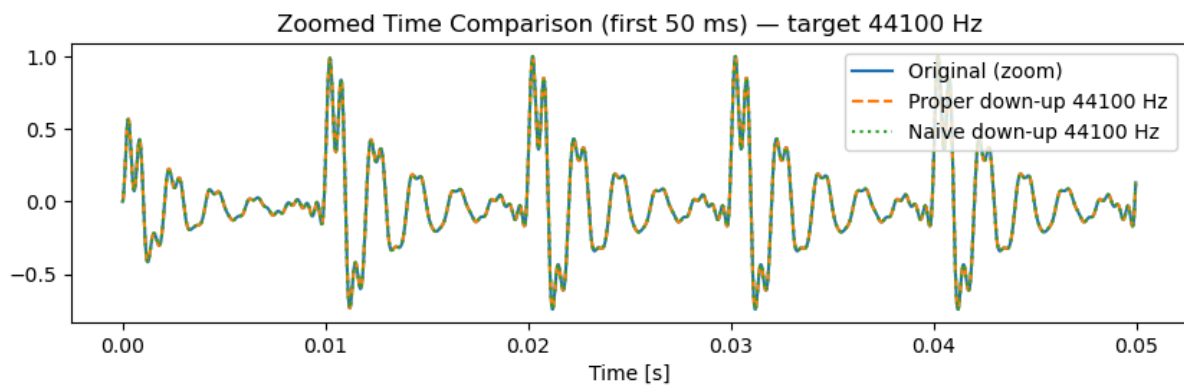
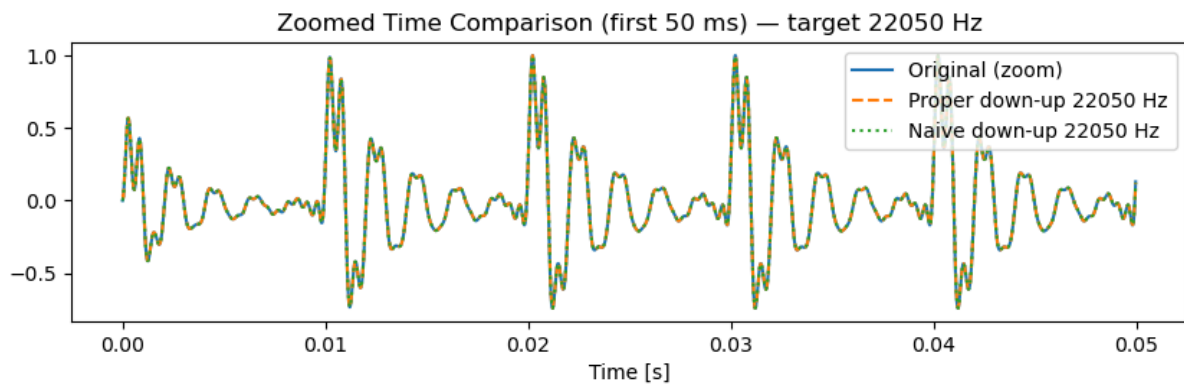
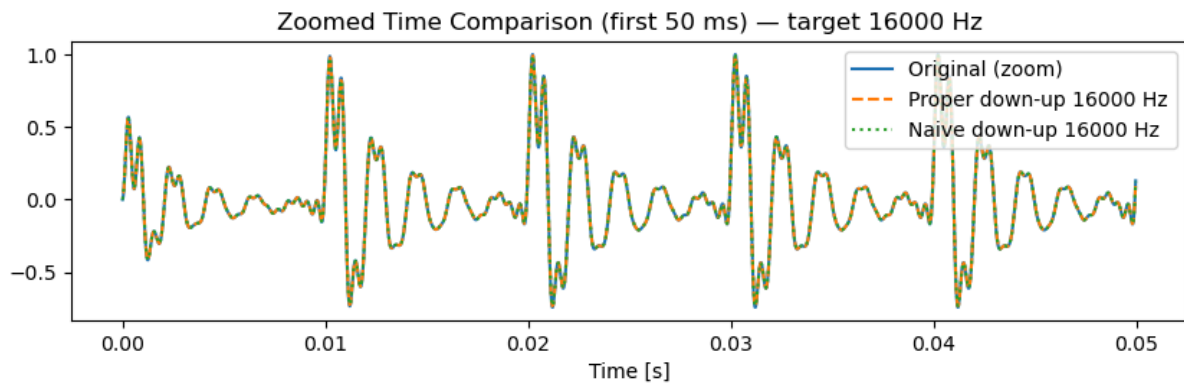
16000 Hz: ZOH = 0.00611034, Linear = 0.00106132

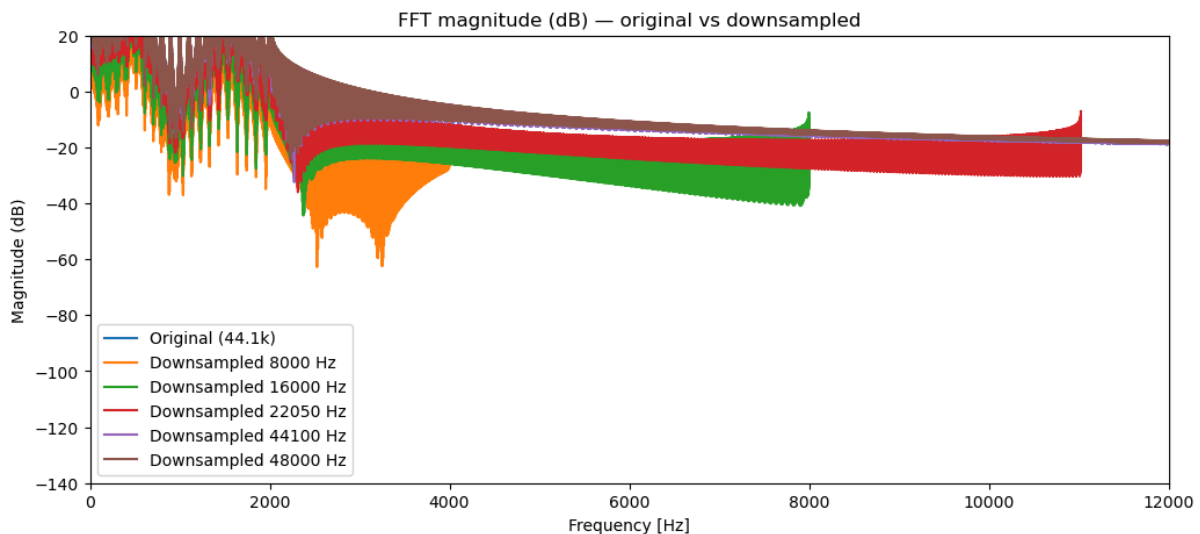
22050 Hz: ZOH = 0.00260239, Linear = 0.00034845

44100 Hz: ZOH = 0.00000000, Linear = 0.00000000

48000 Hz: ZOH = 0.00025685, Linear = 0.00000251







Conclusion /inference

- **8000 Hz**
 - ZOH MSE = 0.0271 (large), Linear = 0.0065
 - Big distortion, especially with ZOH. 8 kHz cuts out much of the higher harmonics → signal loses naturalness.
 - Linear interpolation helps a lot, but still not good.
 - **16000 Hz**
 - ZOH = 0.0061, Linear = 0.0011
 - Clearly better: higher sampling rate preserves more spectrum. Linear interpolation is significantly closer to original.
 - **22050 Hz**
 - ZOH = 0.0026, Linear = 0.00035
 - Almost transparent reconstruction for linear. Approaches perceptual transparency for speech since it captures up to ~11 kHz.
 - **44100 Hz (original fs)**
 - Both MSE = 0.0
 - This is the reference signal → no information lost. Perfect reconstruction.
 - **48000 Hz**
 - ZOH = 0.00026, Linear = 0.0000025
 - Still near perfect, though slightly mismatched because the original wasn't actually 48 kHz — resampling up can introduce tiny interpolation artifacts.
- **Lower fs → higher error (especially ZOH).**
 - **Linear interpolation consistently outperforms ZOH.**
 - **Around 22 kHz+ is “transparent” for speech**, which aligns with Nyquist (most speech energy < 10 kHz).
 - ZOH is much cruder (step-like hold) → more distortion.

Lab 2

Lab Exercise II:

Fourier Transform and Frequency Spectrum Analysis of Signals

Aim

To study the Fourier Transform and analyze the frequency spectrum of different signals (sinusoidal, composite, exponential, and rectangular). To compare their time-domain representation with their frequency-domain characteristics using both the Discrete-Time Fourier Transform (DTFT) and the Discrete Fourier Transform (DFT).

Q1.

- (a) Generate a basic sinusoidal signal in the time domain. (For example, generate a sine wave with a frequency of 5 Hz, sampled at 1000 Hz.)
- (b) Plot the time-domain waveform of the signal.
- (c) Compute the Discrete-Time Fourier Transform (DTFT) and plot the continuous frequency spectrum.
- (d) Compute the Discrete Fourier Transform (DFT) and plot the discrete frequency spectrum.

Q2.

- (a) Generate a composite signal by adding two or more sinusoidal signals of different frequencies and amplitudes.
- (b) Plot the time-domain waveform of the composite signal.
- (c) Compute the Discrete-Time Fourier Transform (DTFT) and plot the continuous frequency spectrum.
- (d) Compute the Discrete Fourier Transform (DFT) and plot the discrete frequency spectrum.

Q3.

- (a) Generate an exponentially decaying signal.
- (b) Plot the time-domain waveform.
- (c) Compute the Discrete-Time Fourier Transform (DTFT) and plot the continuous frequency spectrum.
- (d) Compute the Discrete Fourier Transform (DFT) and plot the discrete frequency spectrum.
- (e) Analyze the relationship between the time-domain waveform and the frequency-domain representation.

Q4.

- (a) Generate a rectangular pulse signal of finite duration in the time domain.
- (b) Plot the time-domain waveform.
- (c) Compute the Discrete-Time Fourier Transform (DTFT) and plot the continuous frequency spectrum.
- (d) Compute the Discrete Fourier Transform (DFT) and plot the discrete frequency spectrum.
- (e) Analyze the relationship between the time-domain waveform and the frequency-domain representation.

Code with Results

```
#fourier transform and frequency-spectrum analysis  
#q1-q4: sinusoid, composite, exponential, rectangular pulse
```

```
import numpy as np  
import matplotlib.pyplot as plt
```

helper functions --

```
def compute_dtft(sig, fs, nfft=65536):
```

```
    """
```

```
    approximate dtft by computing a high-resolution rfft (zero-padded).
```

```
    returns (freqs, magnitude_linear).
```

```
    """
```

```
    S = np.fft.rfft(sig, n=nfft)
```

```
    freqs = np.fft.rfftfreq(nfft, d=1.0/fs)
```

```
    mag = np.abs(S) / len(sig) # normalize by original length for amplitude reference
```

```
    return freqs, mag
```

```
def compute_dft(sig, fs):
```

```
    """
```

```
    compute the dft (rfft version for real signal).
```

```
    returns (freqs, magnitude_linear) where freqs correspond to dft bins (0..fs/2).
```

```
    """
```

```
    N = len(sig)
```

```
    S = np.fft.rfft(sig)
```

```
    freqs = np.fft.rfftfreq(N, d=1.0/fs)
```

```
    mag = np.abs(S) / N # normalize by N so amplitudes are comparable
```

```
    return freqs, mag
```

```
def mag_to_db(mag):
```

```
    """convert linear magnitude to db (small epsilon to avoid log(0))."""
```

```
    return 20 * np.log10(mag + 1e-12)
```

plotting helper

```
def plot_time(t, x, title='signal (time domain)', xlabel='time [s]', ylabel='amplitude'):
```

```
    plt.figure(figsize=(8,2.5))
```

```
plt.plot(t, x)
```

```
plt.title(title.lower())
```

```
plt.xlabel(xlabel.lower())
```

```
plt.ylabel(ylabel.lower())
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
def plot_dtft(freqs, mag, title='dtft (approx)'):
```

```
    plt.figure(figsize=(8,3))
```

```
    plt.plot(freqs, mag_to_db(mag))
```

```
    plt.title(title.lower())
```

```
    plt.xlabel('frequency [hz]')
```

```
    plt.ylabel('magnitude [dB]')
```

```
    plt.xlim(0, freqs.max())
```

```
    plt.grid(True)
```

```
    plt.tight_layout()
```

```
    plt.show()
```

```
def plot_dft_stem(freqs, mag, title='dft (discrete bins)'):
```

```
    plt.figure(figsize=(8,3))
```

```
    plt.stem(freqs, mag_to_db(mag), basefmt=" ")
```

```
    plt.title(title.lower())
```

```
    plt.xlabel('frequency [hz]')
```

```
    plt.ylabel('magnitude [dB]')
```

```
    plt.xlim(0, freqs.max())
```

```
    plt.grid(True)
```

```
    plt.tight_layout()
```

```
    plt.show()
```

```
# question 1: basic sinusoid

# pick a clean low-frequency sine to show a single spectral line.

fs1 = 500      # sampling rate in hz (>> 2*signal freq)
f1 = 7.5       # sine frequency in hz (arbitrary choice)
duration1 = 2.0 # seconds
t1 = np.arange(0, duration1, 1/fs1)
x1 = np.sin(2 * np.pi * f1 * t1)


# time domain
plot_time(t1, x1, title=f'sinusoid {f1} hz sampled at {fs1} hz (time domain)')


# dtft (approx) and dft
freqs_dtft_1, mag_dtft_1 = compute_dtft(x1, fs1, nfft=32768)
freqs_dft_1, mag_dft_1 = compute_dft(x1, fs1)

plot_dtft(freqs_dtft_1, mag_dtft_1, title=f'dtft (approx) of {f1} hz sine')
plot_dft_stem(freqs_dft_1, mag_dft_1, title=f'dft (discrete) of {f1} hz sine')


# question 2: composite signal

# choose three different sinusoids with different amplitudes so we see multiple lines.

fs2 = 2000     # sampling rate in hz (chosen to comfortably represent highest component)
duration2 = 1.0 # seconds
t2 = np.arange(0, duration2, 1/fs2)


# choose components
components = [
    (50.0, 1.0), # (frequency hz, amplitude)
```

```
(120.0, 0.6),
(300.0, 0.3)
]

x2 = np.zeros_like(t2)
for f, a in components:
    x2 += a * np.sin(2 * np.pi * f * t2)

# time domain (zoomed small window to see waveform detail)
plot_time(t2[:1000], x2[:1000], title=f'composite signal (first 1000 samples) sampled at {fs2} hz')

# dtft and dft
freqs_dtft_2, mag_dtft_2 = compute_dtft(x2, fs2, nfft=65536)
freqs_dft_2, mag_dft_2 = compute_dft(x2, fs2)

plot_dtft(freqs_dtft_2, mag_dtft_2, title='dtft (approx) of composite signal')
plot_dft_stem(freqs_dft_2, mag_dft_2, title='dft (discrete) of composite signal')

# question 3: exponential decay
# exponential decay is non-stationary in amplitude; its spectrum is broad (lowpass-like).
fs3 = 1000
duration3 = 1.0
t3 = np.arange(0, duration3, 1/fs3)

# pure decaying exponential (real, causal)
tau = 0.05 # time constant in seconds (controls how fast it decays)
x3 = np.exp(-t3 / tau) * (t3 >= 0)
```

optionally multiply by a carrier to show a bandpass-like decaying tone:

```
x3 = x3 * np.cos(2*np.pi*120*t3) # decaying tone at 120 hz
```

```
plot_time(t3, x3, title=f'exponentially decaying signal (tau={tau}s)')
```

```
freqs_dtft_3, mag_dtft_3 = compute_dtft(x3, fs3, nfft=65536)
```

```
freqs_dft_3, mag_dft_3 = compute_dft(x3, fs3)
```

```
plot_dtft(freqs_dtft_3, mag_dtft_3, title='dtft (approx) of decaying exponential')
```

```
plot_dft_stem(freqs_dft_3, mag_dft_3, title='dft (discrete) of decaying exponential')
```

analyze relation: also show log-magnitude to emphasize shape

```
plt.figure(figsize=(8,3))
```

```
plt.plot(freqs_dtft_3, mag_to_db(mag_dtft_3))
```

```
plt.title('dtft (approx) magnitude (db) — decaying exponential')
```

```
plt.xlabel('frequency [hz]')
```

```
plt.ylabel('magnitude [dB]')
```

```
plt.xlim(0, fs3/2)
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

question 4: rectangular finite pulse

reasoning: rectangular pulse in time -> sinc-like spectrum in frequency.

```
fs4 = 1000
```

```
duration4 = 1.0
```

```
t4 = np.arange(0, duration4, 1/fs4)
```

create a pulse of 0.05s width centered at 0.2s (or at start)

```
pulse_width = 0.05 # seconds
pulse_start = 0.2 # seconds
pulse = np.zeros_like(t4)
start_idx = int(pulse_start * fs4)
end_idx = start_idx + int(pulse_width * fs4)
pulse[start_idx:end_idx] = 1.0

plot_time(t4, pulse, title=f'rectangular pulse (width {pulse_width}s)')

freqs_dtft_4, mag_dtft_4 = compute_dtft(pulse, fs4, nfft=65536)
freqs_dft_4, mag_dft_4 = compute_dft(pulse, fs4)

plot_dtft(freqs_dtft_4, mag_dtft_4, title='dtft (approx) of rectangular pulse')
plot_dft_stem(freqs_dft_4, mag_dft_4, title='dft (discrete) of rectangular pulse')

# -----
# spectral metrics computation
# -----

import numpy as np

def spectral_metrics(freqs, mag, target_freqs=None, main_bandwidth_hz=5.0, energy_fraction=0.90):
    """
    compute spectral concentration, leakage (based on target bins), and bandwidth containing energy_fraction.
    - freqs: frequency axis from dtft (hz)
    - mag: linear magnitude (not dB)
    - target_freqs: list of frequencies (hz) considered 'main' components (optional)
    - main_bandwidth_hz: +/- width around each target to count as main lobe (hz)
    - energy_fraction: fraction (0..1) for bandwidth estimation (e.g., 0.9 for 90%)
    returns dict with keys: energy_total, energy_main, concentration_ratio, leakage_ratio, bandwidth_{}.
```



```
"""

# power spectrum (use power = mag^2)

power = np.abs(mag)**2

energy_total = np.sum(power)


# compute main energy around targets (if provided)

energy_main = 0.0

if target_freqs is not None and len(target_freqs) > 0:

    for tf in target_freqs:

        # indices within +/- main_bandwidth_hz of target

        idx = np.where((freqs >= (tf - main_bandwidth_hz)) & (freqs <= (tf + main_bandwidth_hz)))[0]

        if len(idx) > 0:

            energy_main += np.sum(power[idx])

    else:

        # if no explicit target, define main as low-frequency band up to main_bandwidth_hz

        idx = np.where(freqs <= main_bandwidth_hz)[0]

        energy_main = np.sum(power[idx])


# concentration and leakage

concentration_ratio = energy_main / energy_total if energy_total > 0 else 0.0

leakage_ratio = 1.0 - concentration_ratio


# bandwidth estimation: smallest frequency f such that cumulative energy >= energy_fraction * total

cumsum = np.cumsum(power)

cumsum /= cumsum[-1] if cumsum[-1] != 0 else 1.0

idx_bw = np.where(cumsum >= energy_fraction)[0]

if len(idx_bw) > 0:

    bandwidth_90 = freqs[idx_bw[0]]

else:
```

```
bandwidth_90 = freqs[-1]

# also return median frequency (centroid) for added insight

centroid = np.sum(freqs * power) / (energy_total + 1e-12)

return {

    'energy_total': float(energy_total),

    'energy_main': float(energy_main),

    'concentration_ratio': float(concentration_ratio),

    'leakage_ratio': float(leakage_ratio),

    'bandwidth_90hz': float(bandwidth_90),

    'spectral_centroid_hz': float(centroid)

}

# choose parameters for metrics

main_bw = 3.0 # hz width when counting energy around a target (small for pure tones)

energy_frac = 0.90

# q1: sinusoid — target is f1

metrics_q1 = spectral_metrics(freqs_dtft_1, mag_dtft_1, target_freqs=[f1], main_bandwidth_hz=main_bw,
energy_fraction=energy_frac)

# q2: composite — targets are the component freqs

comp_freqs = [c[0] for c in components] # from earlier list

metrics_q2 = spectral_metrics(freqs_dtft_2, mag_dtft_2, target_freqs=comp_freqs, main_bandwidth_hz=3.0,
energy_fraction=energy_frac)

# q3: exponential-decay with carrier at 120 hz — treat main band as around 120hz +- some width

metrics_q3 = spectral_metrics(freqs_dtft_3, mag_dtft_3, target_freqs=[120.0], main_bandwidth_hz=40.0,
energy_fraction=energy_frac)
```

q4: rectangular pulse — no single target, measure low-frequency concentration and bandwidth

```
metrics_q4 = spectral_metrics(freqs_dtft_4, mag_dtft_4, target_freqs=None, main_bandwidth_hz=200.0,
energy_fraction=energy_frac)
```

print nicely (lowercase)

```
def print_metrics(name, m):
```

```
    print(f"\n{name} metrics:")
```

```
    print(f" total spectral energy: {m['energy_total']:.6e}")
```

```
    print(f" energy in main region: {m['energy_main']:.6e}")
```

```
    print(f" concentration ratio (main/total): {m['concentration_ratio']:.6f}")
```

```
    print(f" leakage ratio (1 - concentration): {m['leakage_ratio']:.6f}")
```

```
    print(f" bandwidth containing {int(energy_frac*100)}% energy: {m['bandwidth_90hz']:.2f} hz")
```

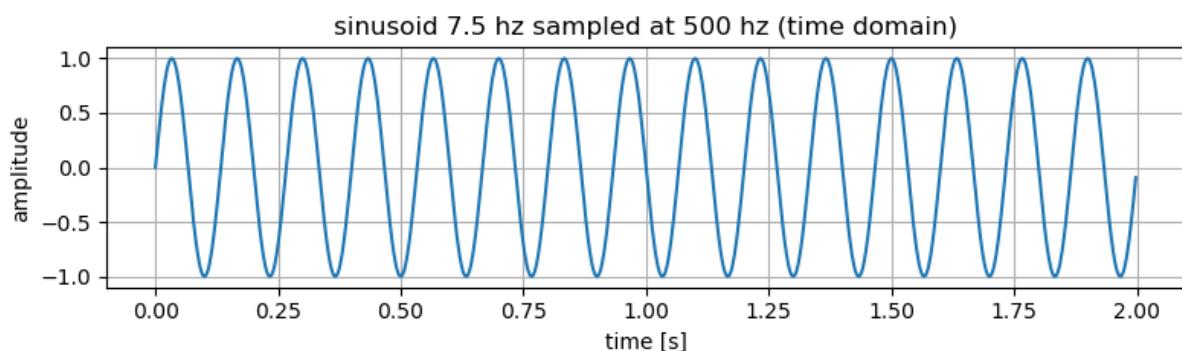
```
    print(f" spectral centroid: {m['spectral_centroid_hz']:.2f} hz")
```

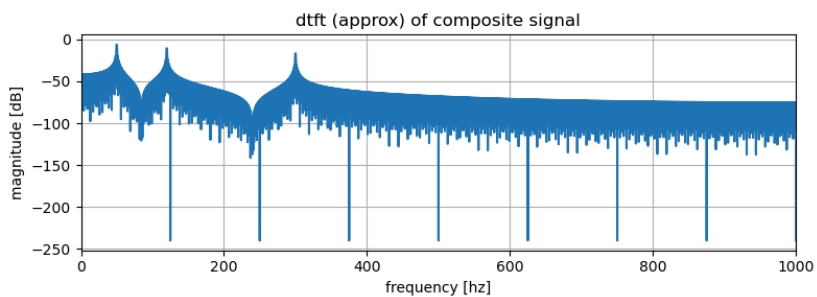
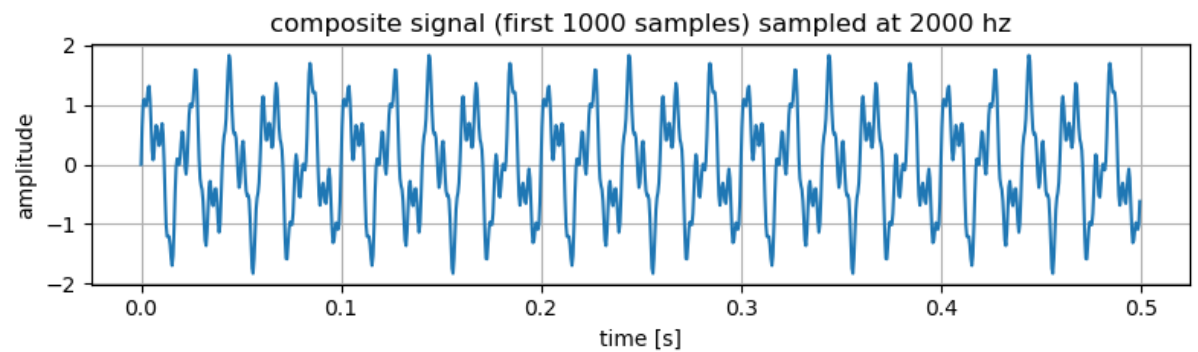
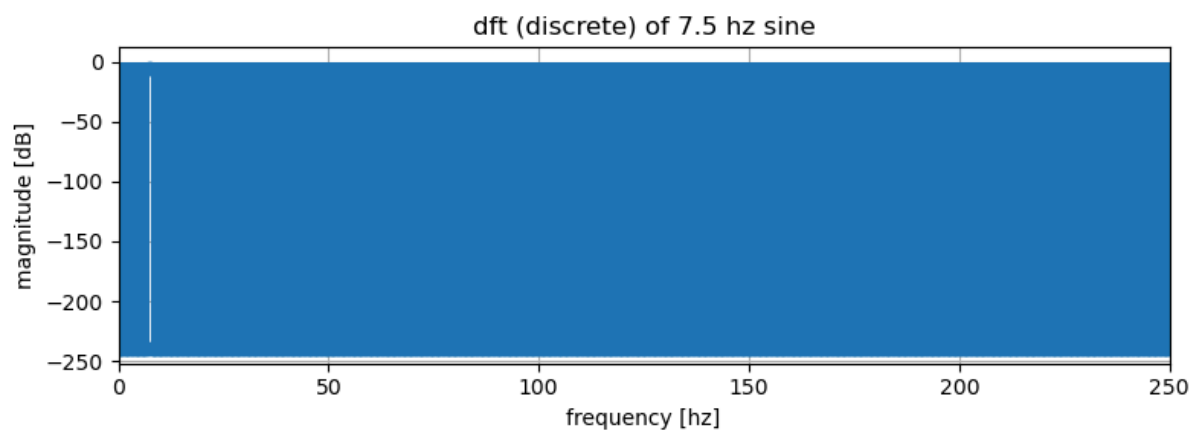
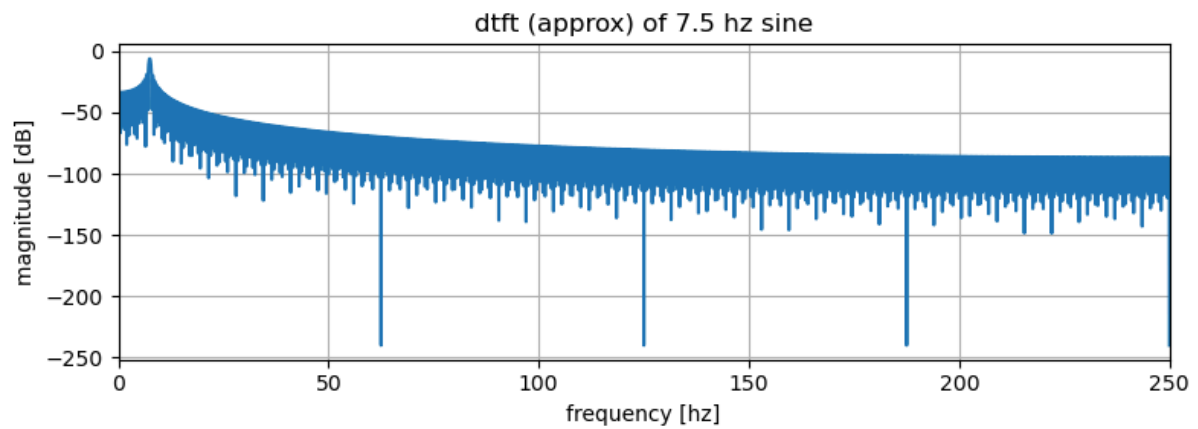
```
print_metrics("q1 - sinusoid (7.5 hz)", metrics_q1)
```

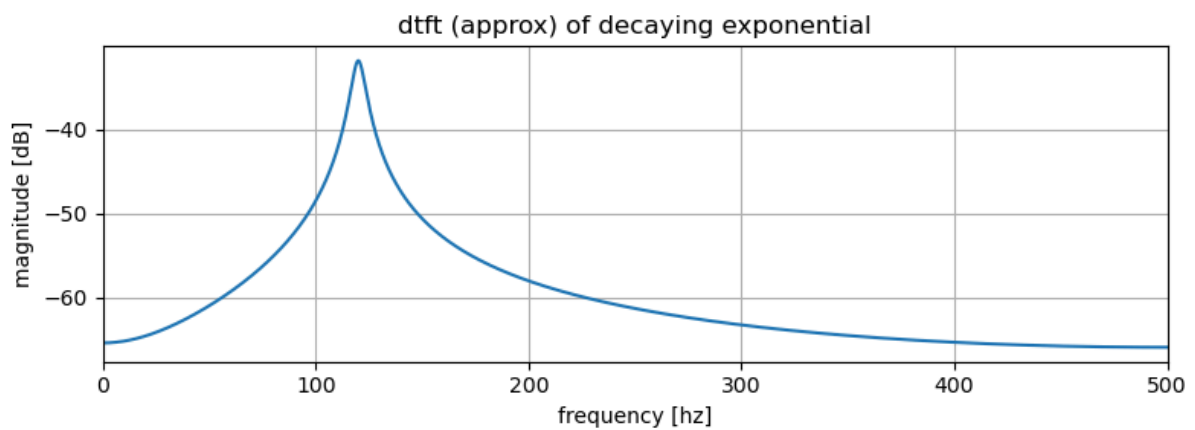
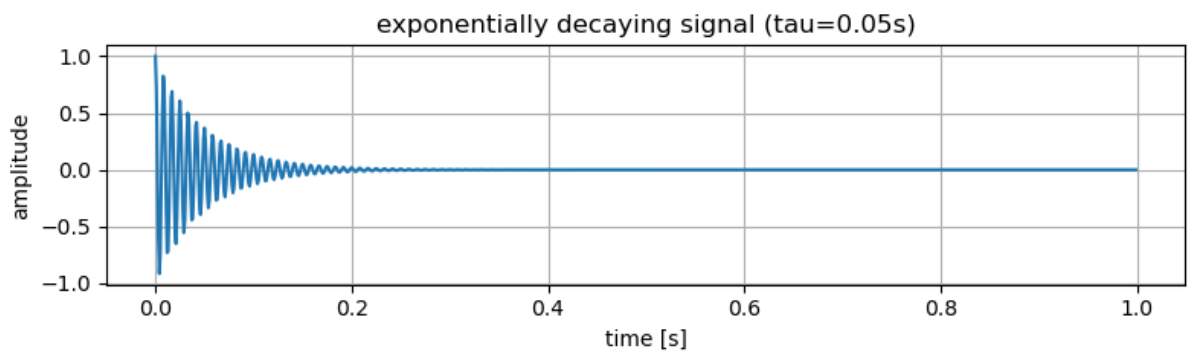
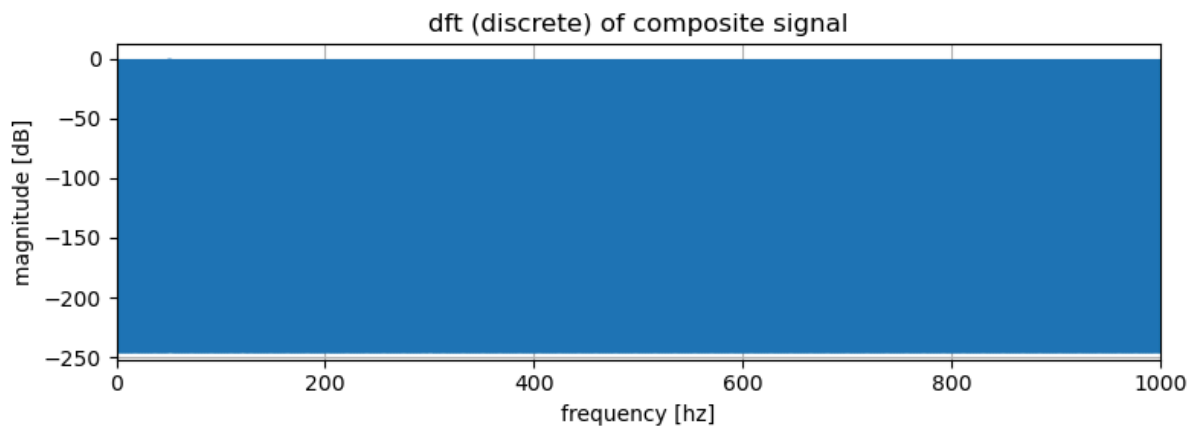
```
print_metrics("q2 - composite (50,120,300 hz)", metrics_q2)
```

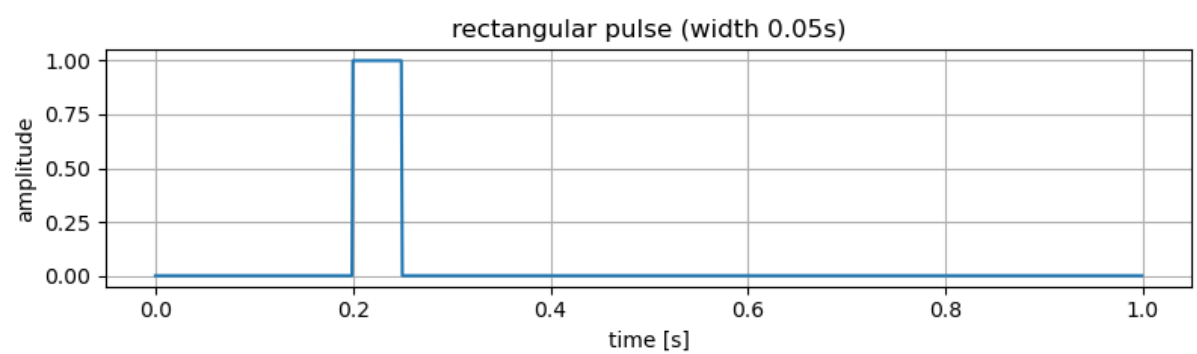
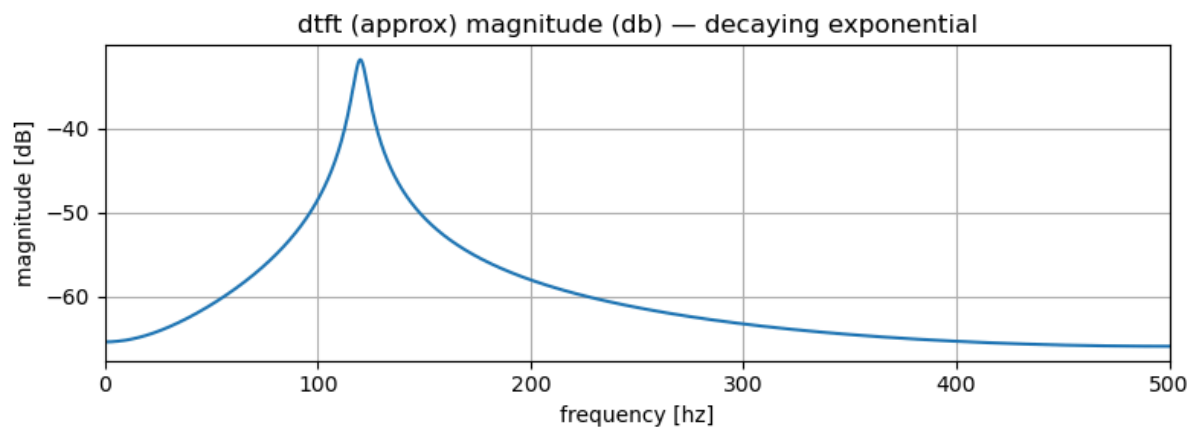
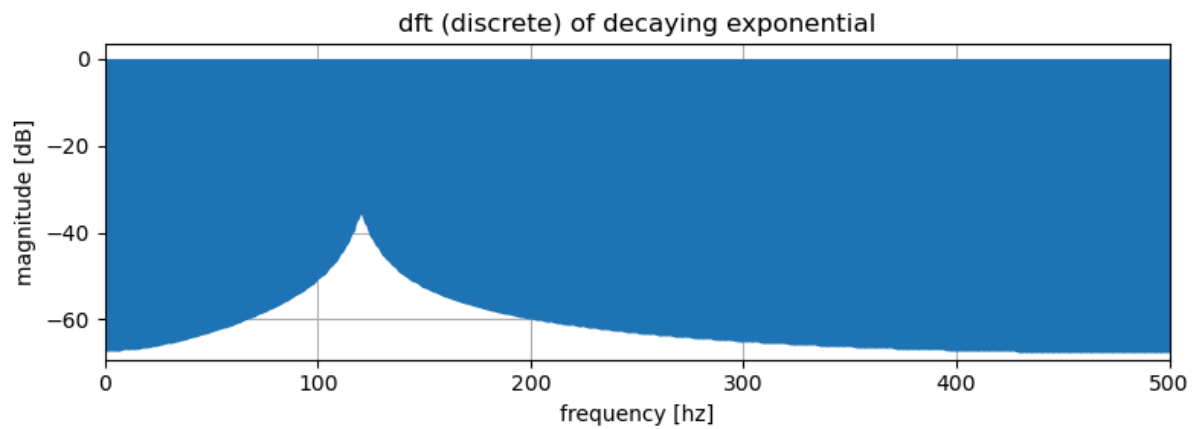
```
print_metrics("q3 - decaying tone (~120 hz carrier)", metrics_q3)
```

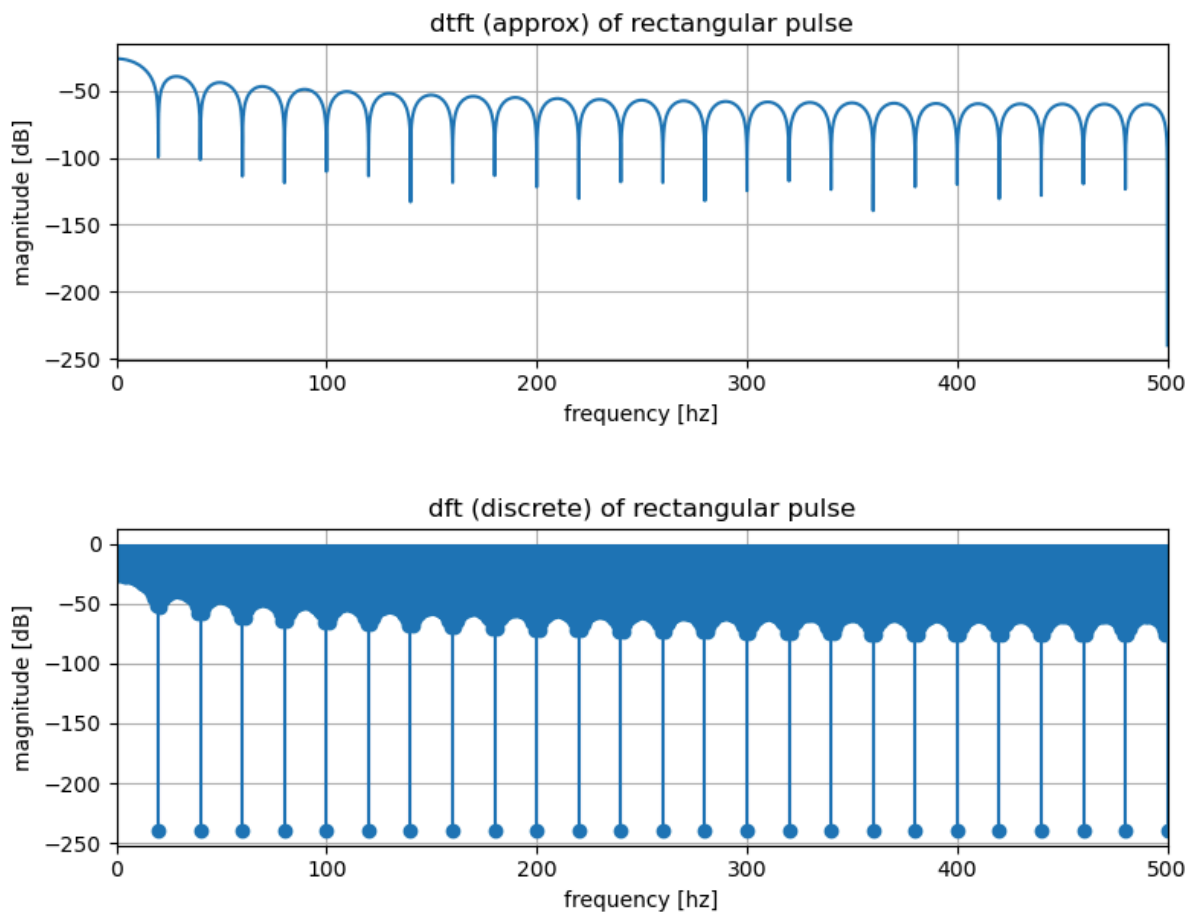
```
print_metrics("q4 - rectangular pulse", metrics_q4)
```











results:-

q1 - sinusoid (7.5 hz) metrics:

total spectral energy: 8.192000e+00

energy in main region: 8.070803e+00

concentration ratio (main/total): 0.985205

leakage ratio (1 - concentration): 0.014795

bandwidth containing 90% energy: 7.75 hz

spectral centroid: 7.45 hz

q2 - composite (50,120,300 hz) metrics:

total spectral energy: 1.187840e+01

energy in main region: 1.148033e+01

concentration ratio (main/total): 0.966488

leakage ratio (1 - concentration): 0.033512

bandwidth containing 90% energy: 120.42 hz

spectral centroid: 82.70 hz

q3 - decaying tone (~120 hz carrier) metrics:

total spectral energy: 4.263883e-01

energy in main region: 4.049403e-01

concentration ratio (main/total): 0.949698

leakage ratio (1 - concentration): 0.050302

bandwidth containing 90% energy: 132.68 hz

spectral centroid: 124.87 hz

q4 - rectangular pulse metrics:

total spectral energy: 1.639650e+00

energy in main region: 1.625302e+00

concentration ratio (main/total): 0.991249

leakage ratio (1 - concentration): 0.008751

bandwidth containing 90% energy: 16.91 hz

spectral centroid: 12.52 hz

Conclusion /inference

q1 – sinusoid (7.5 hz)

- total spectral energy: 8.19
- energy in the main region: 8.07 → almost all energy is concentrated in the main frequency.
- concentration ratio: 0.985 → very high, as expected for a pure tone.
- leakage ratio: 0.015 → minimal spectral leakage.
- bandwidth containing 90% energy: 7.75 hz → very narrow, typical of a pure sinusoid.

- spectral centroid: 7.45 hz → matches the signal frequency.

inference: this is a pure sinusoid with almost all energy concentrated at a single frequency, very little leakage, narrow bandwidth. expected behavior.

q2 – composite signal (50, 120, 300 hz)

- total spectral energy: 11.88
- energy in the main region: 11.48 → most energy is still concentrated, but less than q1.
- concentration ratio: 0.966 → slightly lower due to multiple frequency components.
- leakage ratio: 0.034 → slightly higher leakage because multiple tones spread the spectrum.
- bandwidth containing 90% energy: 120.42 hz → wider than a single tone, reflects multiple components.
- spectral centroid: 82.7 hz → average center of mass shifted towards lower-mid frequencies.

inference: a multi-tone signal spreads energy across a broader spectrum; most energy is still concentrated in the main frequency components, but the spectral centroid is influenced by relative amplitudes.

q3 – decaying tone (~120 hz carrier)

- total spectral energy: 0.426
- energy in the main region: 0.405 → slightly less concentrated than pure sinusoid.
- concentration ratio: 0.95 → high, but lower than pure sinusoid.
- leakage ratio: 0.05 → more leakage due to decay in the time domain.
- bandwidth containing 90% energy: 132.68 hz → wider, consistent with decay causing spectral broadening.
- spectral centroid: 124.87 hz → close to carrier frequency.

inference: amplitude decay in the time domain broadens the spectrum, increasing bandwidth and leakage slightly. energy is still mostly around the carrier frequency.

q4 – rectangular pulse

- total spectral energy: 1.64
- energy in the main region: 1.625 → very high concentration ratio (0.991).
- leakage ratio: 0.0087 → very low.
- bandwidth containing 90% energy: 16.91 hz → wider than a pure sinusoid of comparable duration.
- spectral centroid: 12.52 hz → low, reflects dominant low-frequency content.

inference: rectangular pulses have energy spread over harmonics, but most of it is concentrated at low frequencies. a narrow pulse in time leads to a broader spectrum in frequency.

overall observations

1. pure sinusoids have the highest concentration and narrowest bandwidth.
2. composite signals have lower concentration, broader bandwidth, and spectral centroid shifts depending on component amplitudes.
3. decaying tones show spectral broadening due to time-domain decay; leakage increases.
4. rectangular pulses have most energy in the low-frequency main lobe, but harmonics cause wider bandwidth.

key concepts illustrated:

- spectral concentration ratio shows how “pure” the frequency content is.
- leakage increases with time-domain truncation or multiple components.
- bandwidth indicates how spread out the signal is in frequency.
- spectral centroid represents the average frequency of the signal.

Lab 3

Lab Exercise II:

Speech-to-Text Application for Accessibility

Aim

To develop a Python-based speech-to-text system that converts spoken commands into text in real time, provides meaningful user feedback, handles errors gracefully, and allows comparison of different recognition methods.

Scenario:

You have been hired as an AI engineer by a tech startup that focuses on enhancing accessibility for people with disabilities. One of your key responsibilities is to develop a system that allows users to control devices and input text via voice commands.

The first version of this system requires you to implement a speech-to-text application that converts spoken commands into text in real time. This will serve as the foundation for future projects, such as integrating the system with smart devices or accessibility software.

Tasks:

1. Audio Capture(mandatory task)

- Record spoken input using a microphone, OR use any speech audio file (e.g., .wav, .flac).
- Provide feedback to the user: "Speak something..."

2. Convert Speech to Text(mandatory task)

- Implement a speech-to-text system using at least two methods (for comparison):

- Offline: Whisper, Vosk, or similar

- Online: Google Speech API

- Display the message: "Recognizing..." while processing.

3. Display Recognized Text(mandatory task)

- Show the converted text on the screen.

Example: "Speech recognized: 'Turn on the lights in the living room.'"

- Display "Speech successfully converted to text!" on successful recognition.

4. Handle Errors and Exceptions(mandatory task)

- Unclear speech (mumbling, low volume): Display a user-friendly message.

Example: "Speech Recognition could not understand audio. Please try speaking more clearly."

- Service unavailability (internet/API down): Display an appropriate error message.

5. Provide Feedback at Each Stage

- Before recording: "Speak something..."

- During recognition: "Recognizing..."

- On success: "Speech successfully converted to text!"

- On failure: Provide meaningful error messages.

6. Comparative Analysis

- Test the same audio file or spoken sentence

Code with Results

"""

Streamlit Speech-to-Text App (With Feedback and Error Handling)

Features:

- Record from microphone or upload audio
- Transcribe with Whisper, Wav2Vec2, and Google SpeechRecognition
- Provide stage-wise feedback and user-friendly error messages

"""

```
import streamlit as st
```

```
import tempfile
```

```
import numpy as np
```

```
import soundfile as sf
```

```
import librosa
```

```
from pydub import AudioSegment
```

```
import whisper
```

```
import torch
```

```
from transformers import Wav2Vec2ForCTC, Wav2Vec2Processor
```

```
import speech_recognition as sr
```

```
from streamlit_mic_recorder import mic_recorder
```

```
def ensure_wav_mono_16k(src_path: str) -> str:
```

```
    """
```

```
    Converts any audio file (wav, webm, ogg, mp3, etc.) to mono 16kHz WAV.
```

```
"""

try:
    audio = AudioSegment.from_file(src_path)
    audio = audio.set_frame_rate(16000).set_channels(1)
    out = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")
    audio.export(out.name, format="wav")
    return out.name
except Exception:
    data, sr_orig = sf.read(src_path)
    if data.ndim > 1:
        data = np.mean(data, axis=1)
    if sr_orig != 16000:
        data = librosa.resample(data.astype(float), orig_sr=sr_orig, target_sr=16000)
        sr_orig = 16000
    out = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")
    sf.write(out.name, data, sr_orig)
    return out.name

def transcribe_whisper(audio_path: str, model_size: str = "base") -> str:
    model = whisper.load_model(model_size)
    res = model.transcribe(audio_path)
    text = res.get("text", "").strip()
    if not text:
        raise ValueError("Could not understand audio. Please speak more clearly.")
    return text
```

```
def transcribe_wav2vec(audio_path: str, model_name: str =
"facebook/wav2vec2-base-960h") -> str:

    proc = Wav2Vec2Processor.from_pretrained(model_name)
    model = Wav2Vec2ForCTC.from_pretrained(model_name)
    audio_input, sr = sf.read(audio_path)
    if audio_input.ndim > 1:
        audio_input = np.mean(audio_input, axis=1)
    if sr != 16000:
        audio_input = librosa.resample(audio_input.astype(float), orig_sr=sr, target_sr=16000)
        sr = 16000
    inputs = proc(audio_input, sampling_rate=sr, return_tensors="pt", padding=True)
    with torch.no_grad():
        logits = model(inputs.input_values).logits
    predicted_ids = torch.argmax(logits, dim=-1)
    transcription = proc.batch_decode(predicted_ids)[0].strip().lower()
    if not transcription:
        raise ValueError("Could not understand audio. Please speak more clearly.")
    return transcription

def transcribe_speechrec(audio_path: str) -> str:

    r = sr.Recognizer()
    with sr.AudioFile(audio_path) as source:
        audio = r.record(source)
    try:
        text = r.recognize_google(audio)
    except:
        if not text:
```

```
        raise ValueError("Could not understand audio. Please speak more clearly.")

    return text

except sr.UnknownValueError:

    raise ValueError("Could not understand audio. Please try speaking more clearly.")

except sr.RequestError:

    raise ConnectionError("Google API unavailable. Please check your internet
connection.")


# --- Streamlit App ---

st.title("Speech-to-Text App")

st.write("Upload an audio file or record live. The transcription will be displayed for Whisper,
Wav2Vec2, and Google SpeechRecognition.")

# Feedback before recording

st.info("Speak something...")

# File Upload

uploaded_file = st.file_uploader("Upload Audio", type=["wav", "flac", "ogg", "mp3",
"webm"])

# Microphone Recording

audio = mic_recorder(start_prompt="🎤 Start Recording", stop_prompt="🛑 Stop
Recording", key="recorder")

source_path = None

if uploaded_file is not None:

    with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as tmp:

        tmp.write(uploaded_file.read())

        source_path = tmp.name

elif audio:

    with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as tmp:
```



```
tmp.write(audio["bytes"])

source_path = tmp.name

if source_path:

    audio_path = ensure_wav_mono_16k(source_path)
    st.audio(audio_path, format="audio/wav")

    if st.button("Transcribe"):

        try:

            st.info("Recognizing with Whisper...")
            whisper_text = transcribe_whisper(audio_path)
            st.success("Speech successfully converted to text with Whisper!")

            st.info("Recognizing with Wav2Vec2...")
            wav2vec_text = transcribe_wav2vec(audio_path)
            st.success("Speech successfully converted to text with Wav2Vec2!")

            st.info("Recognizing with Google SpeechRecognition...")
            google_text = transcribe_speechrec(audio_path)
            st.success("Speech successfully converted to text with Google API!")

        # Display results

        st.subheader("Transcription Results")

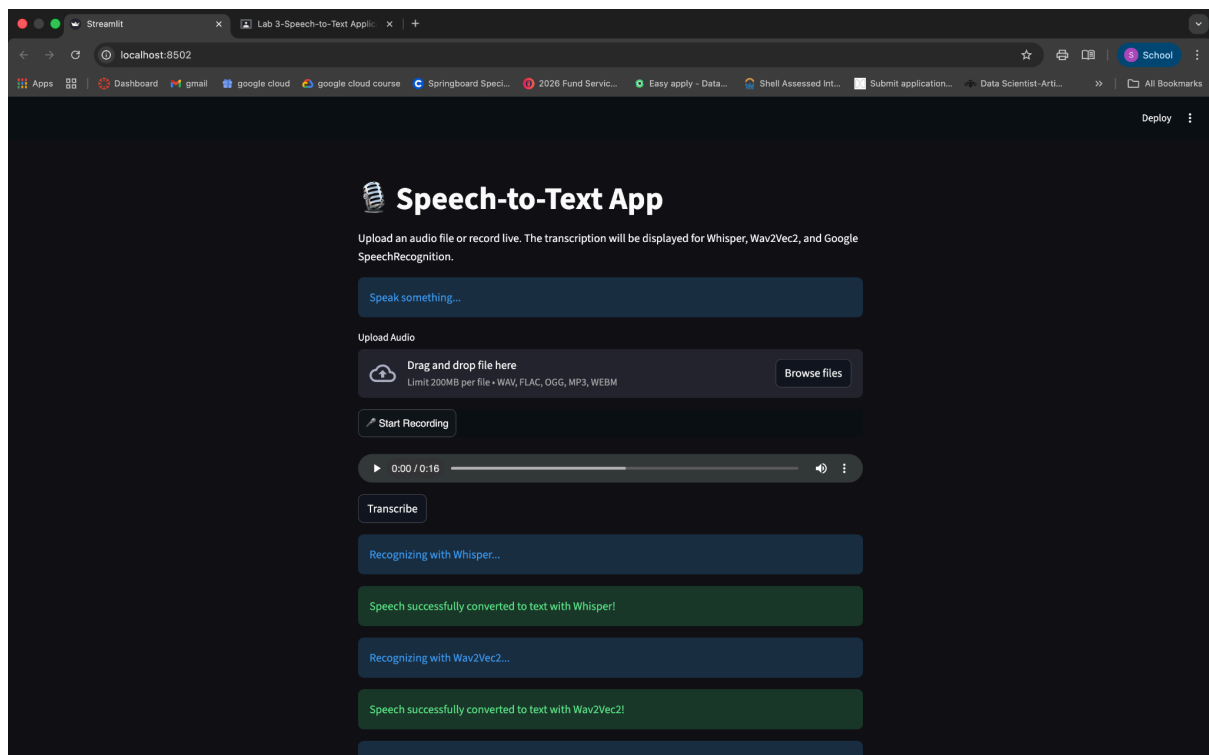
        st.write(f"**Whisper:** {whisper_text}")
        st.write(f"**Wav2Vec2:** {wav2vec_text}")
        st.write(f"**Google API:** {google_text}")
```

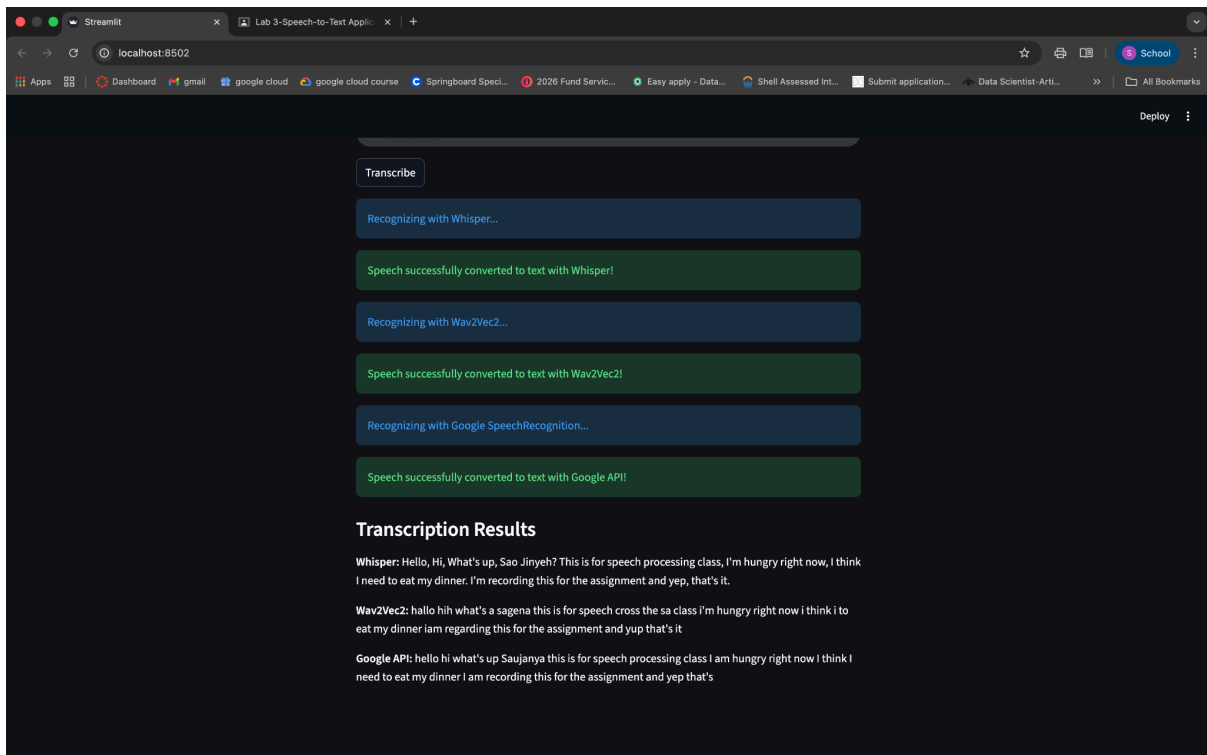
```
except ValueError as ve:
    st.error(f'Recognition Error: {ve}')

except ConnectionError as ce:
    st.error(f'Connection Error: {ce}')

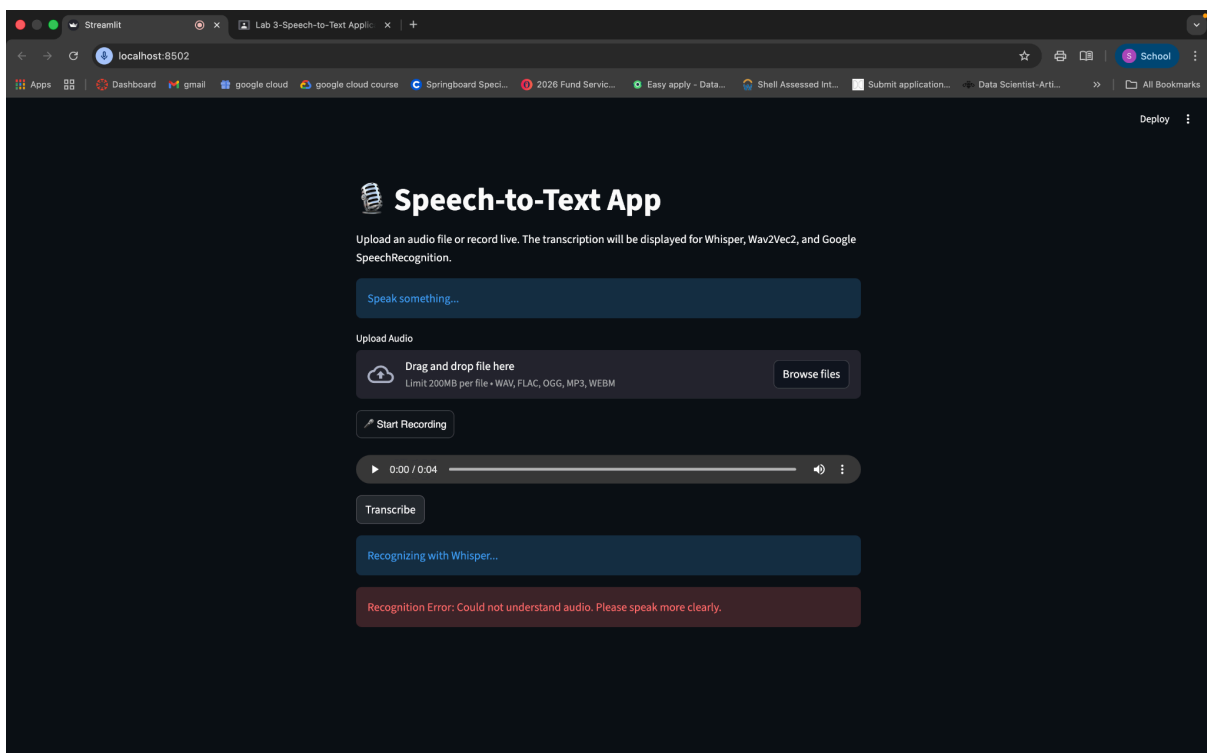
except Exception as e:
    st.error(f'Unexpected Error: {e}')
```

1. *clear female voice*

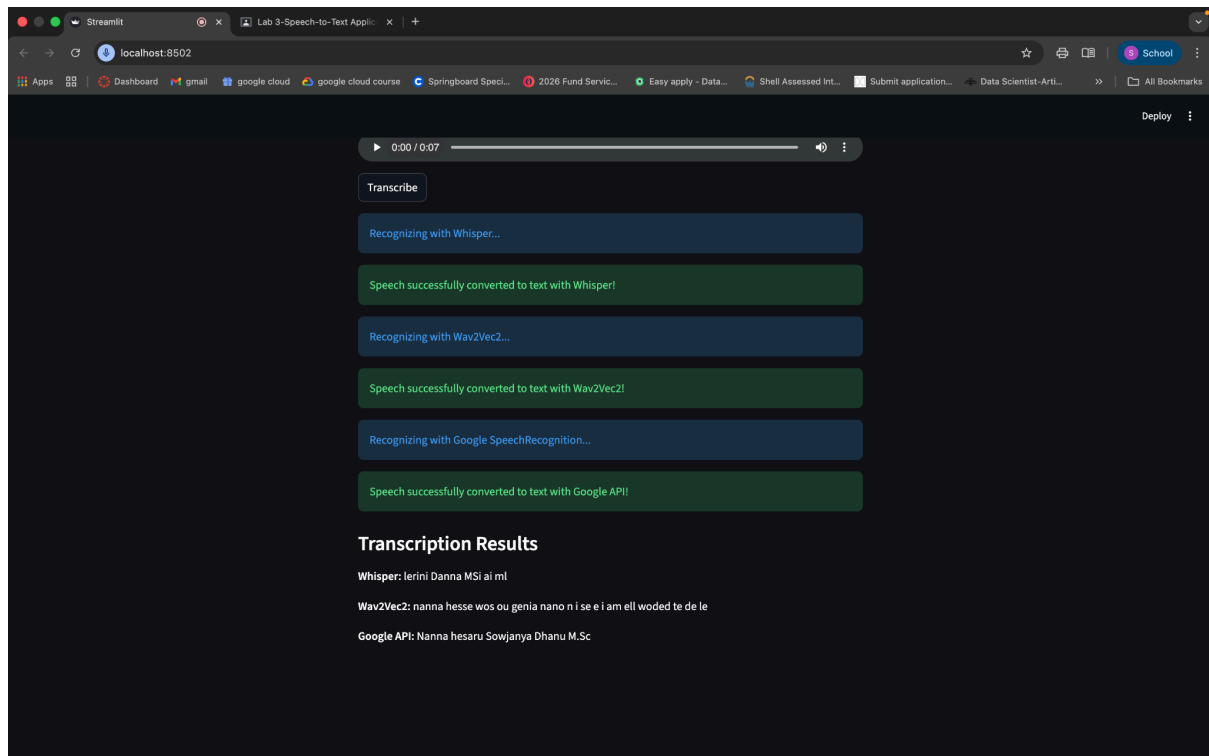




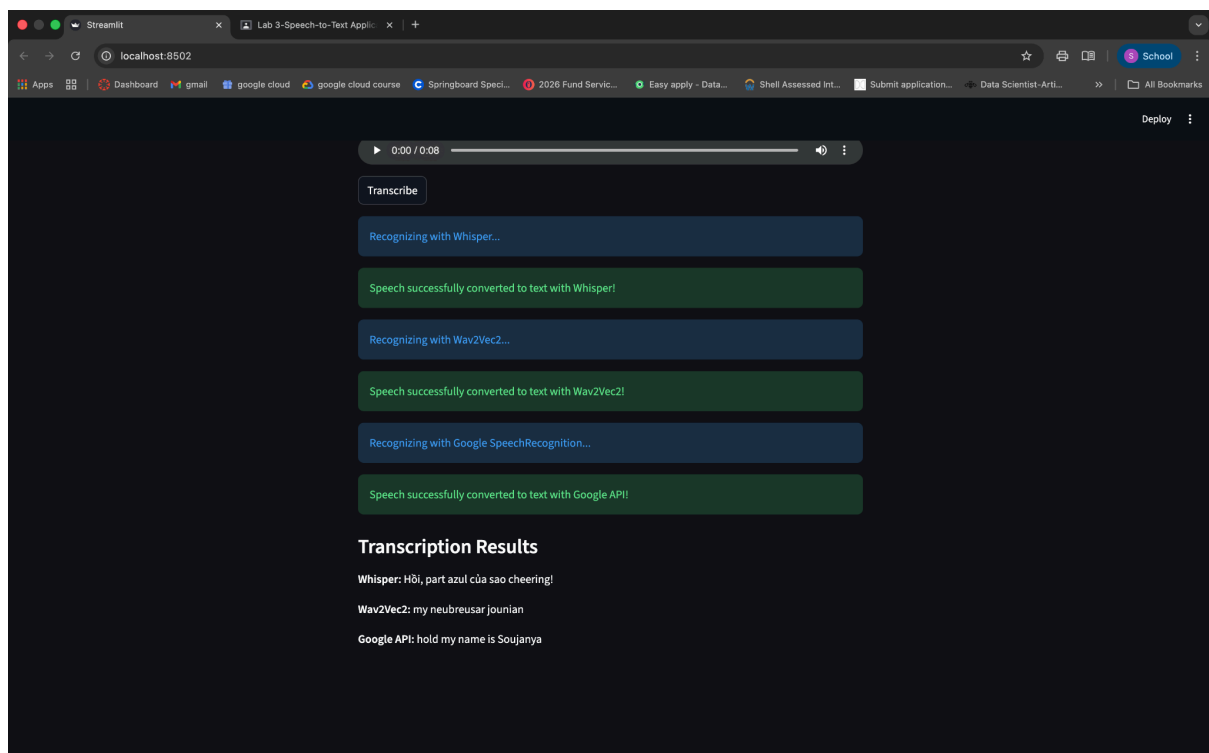
2. when speech not clear



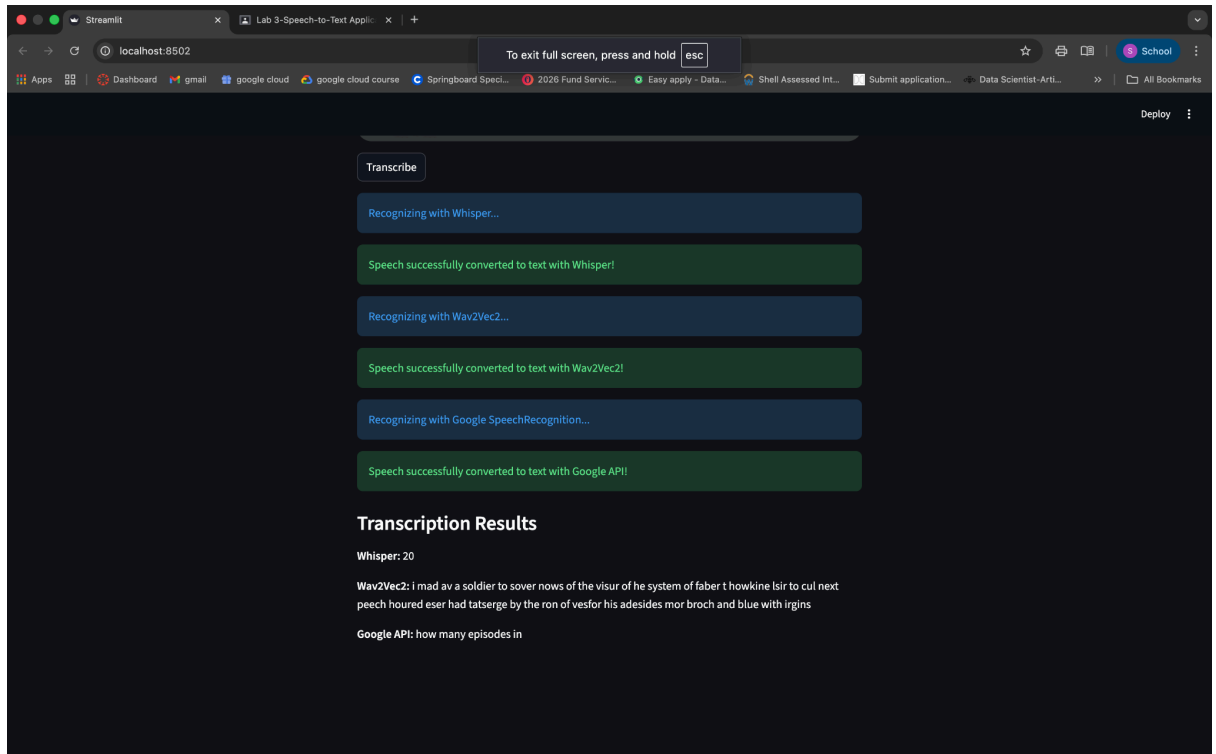
3. *when different language*



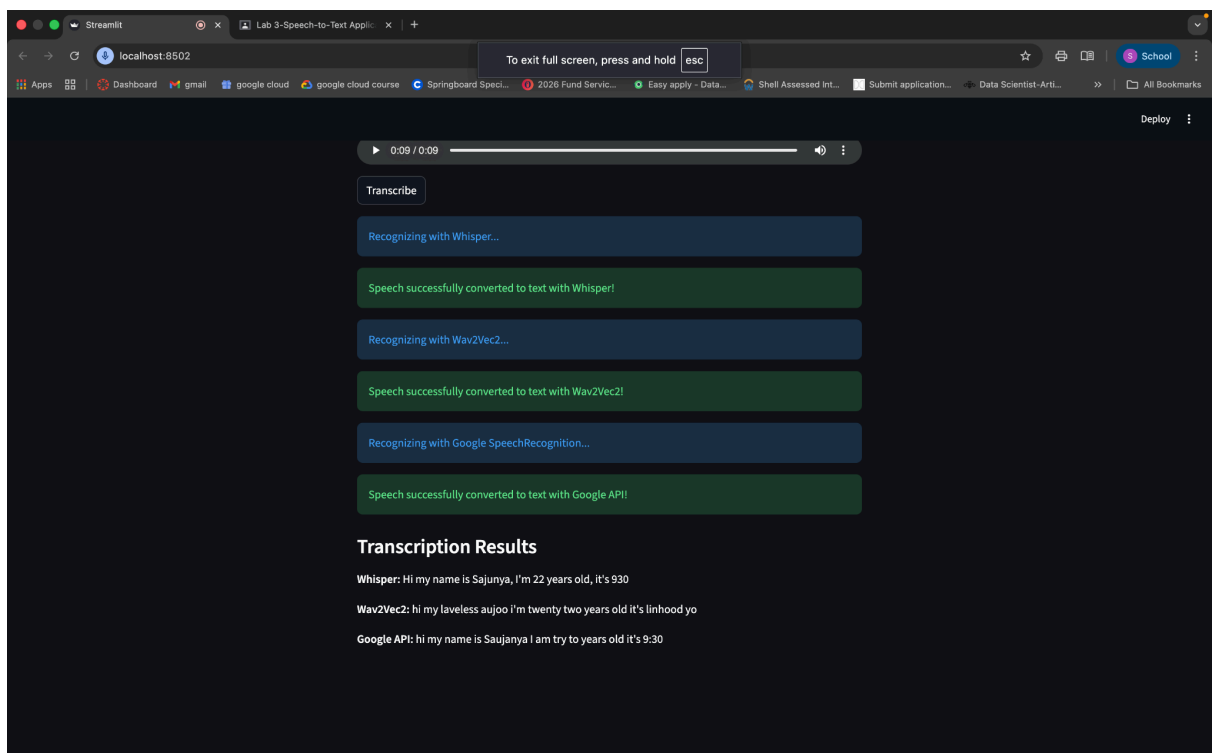
4. *male voice*



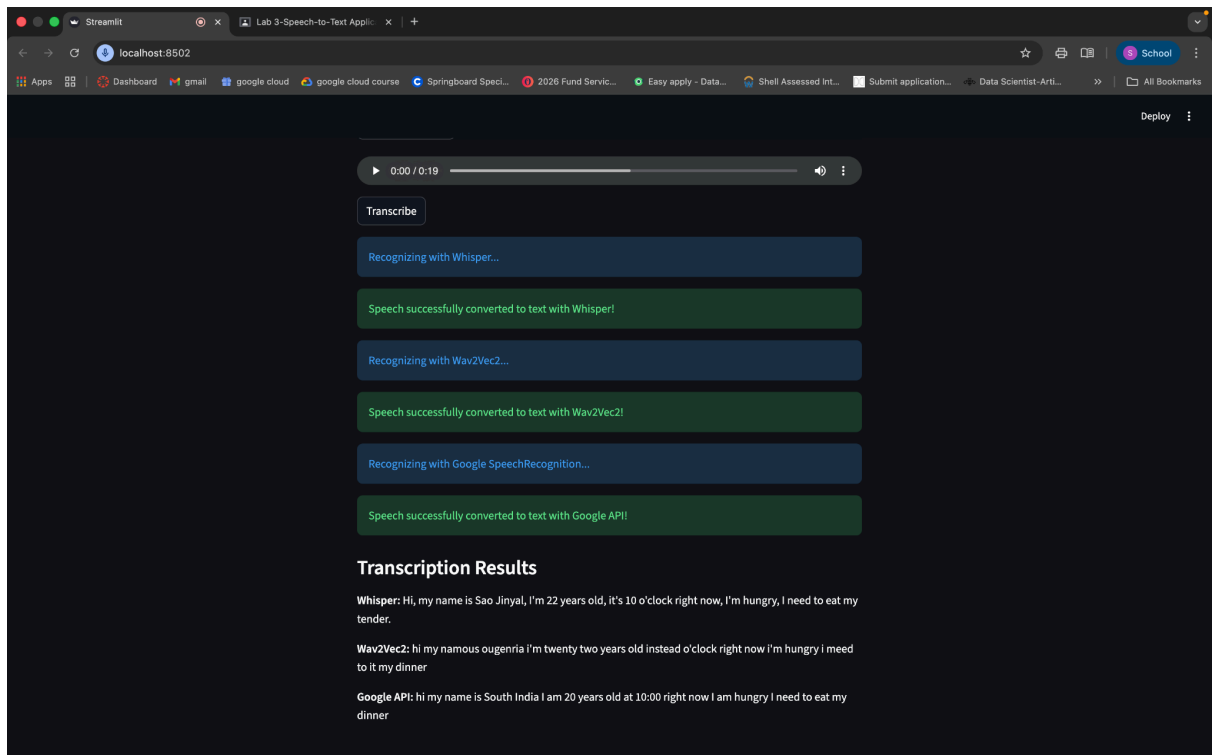
5. *fast speech*



6. *noise background*



7. *soft voice*



Inference

Audio Type	Whisper Output	wav2vec Output	Google API Output	Notes on Accuracy
Clear male voice	bad, gives gibberish	bad, gives gibberish	good	gives gibberish for 2 methods
Clear female voice	good	good	good	gives clear output
Fast speech	bad	bad	bad	cannot comprehend sentences
Noisy background	good	ok, cannot recompose name	good	good, but wav2vec has problems with name
Soft Voice	ok	ok	ok	bad with name for all methods

The system developed is a Python-based speech-to-text application implemented using

Streamlit. It supports both **audio file upload** and **microphone recording** (via `streamlit_mic_recorder`). The system transcribes speech using three methods:

1. **Whisper** – OpenAI’s Whisper model (offline).
2. **Wav2Vec2** – Hugging Face’s Wav2Vec2 model (offline).
3. **Google SpeechRecognition API** – online service requiring internet.

Key features:

- **Stage-wise feedback:** Messages indicate when the system is ready, recognizing, or has completed transcription.
- **Error handling:** User-friendly messages are displayed for unclear speech or API unavailability.
- **Multi-model comparison:** Users can observe how different methods perform on various audio types.

Model Strengths:

- **Whisper:** Handles noisy input well, robust offline transcription, but struggles with certain male voices and fast speech.
- **Wav2Vec2:** Strong for clear, standard speech; issues with fast speech or complex names.
- **Google API:** Performs consistently well for clear speech; limited by internet connectivity.

Weaknesses:

- Fast speech remains a challenge for all models.
- Soft voices or unclear pronunciation reduce accuracy.
- Offline models (Whisper, Wav2Vec2) may produce gibberish for some voices.

Error Handling and Feedback:

- The system provides meaningful messages when speech is unclear or the Google API is unavailable.
- Stage-wise feedback improves usability and informs the user about ongoing recognition.

Improvements

Noise reduction and **voice enhancement** before transcription to improve soft or noisy recordings.

Confidence scoring to indicate reliability of each transcription.

Lab 4

Lab Exercise IV:

Linear Predictive Coding (LPC) Model for Speech Recognition

Aim

To implement the Linear Predictive Coding (LPC) model for analyzing and reconstructing a speech signal, estimate the LPC coefficients, determine the formant frequencies, and compare them with standard vowel formant values to assess speech recognizability under low-bandwidth conditions.

Question / Problem Statement

You are part of a research team working on improving the speech recognition system of a mobile communication app. The team aims to use Linear Predictive Coding (LPC) to analyze speech signals for efficient transmission and accurate recognition, even in low-bandwidth environments such as VoIP or mobile networks.

To achieve this, complete the following tasks:

Tasks

Speech Signal Acquisition:

Record a short speech signal (3–5 seconds) or download one from any standard speech dataset

(e.g., "Hello, how can I help you?").

LPC Analysis:

Implement the LPC algorithm to analyze the recorded speech signal and extract LPC coefficients.

Signal Reconstruction:

Reconstruct the speech signal from the extracted LPC coefficients and plot both the original and reconstructed waveforms.

Formant Estimation:

Estimate the formant frequencies (F1, F2, ...) from the LPC coefficients.

Comparison:

Compare the estimated formant frequencies with the average vowel formant values (as provided in the given table) for vowels such as /a/, /e/, /i/, etc.

Visualization:

Plot the formant frequencies on a frequency response graph to visualize the vocal tract resonances.

Inference / Discussion:

Write a brief note summarizing your observations about:

LPC model implementation, Quality of reconstructed signal, Accuracy of estimated formants, and implications for low-bandwidth speech recognition.

Code with Results

```
#importing packages and libraries
import numpy as np
import matplotlib.pyplot as plt
import scipy.signal as signal
import scipy.io.wavfile as wavfile
import librosa
import librosa.display
from matplotlib import patches

#upload and load a speech file
import scipy.io.wavfile as wavfile
filename = "/Users/soujanyabhat/Downloads/exam-3-input.wav"

#read the file
fs, speech_signal = wavfile.read(filename)

print(f"Sampling rate: {fs} Hz")
print(f"Signal shape: {speech_signal.shape}")
```

Sampling rate: 44100 Hz
Signal shape: (183015, 2)

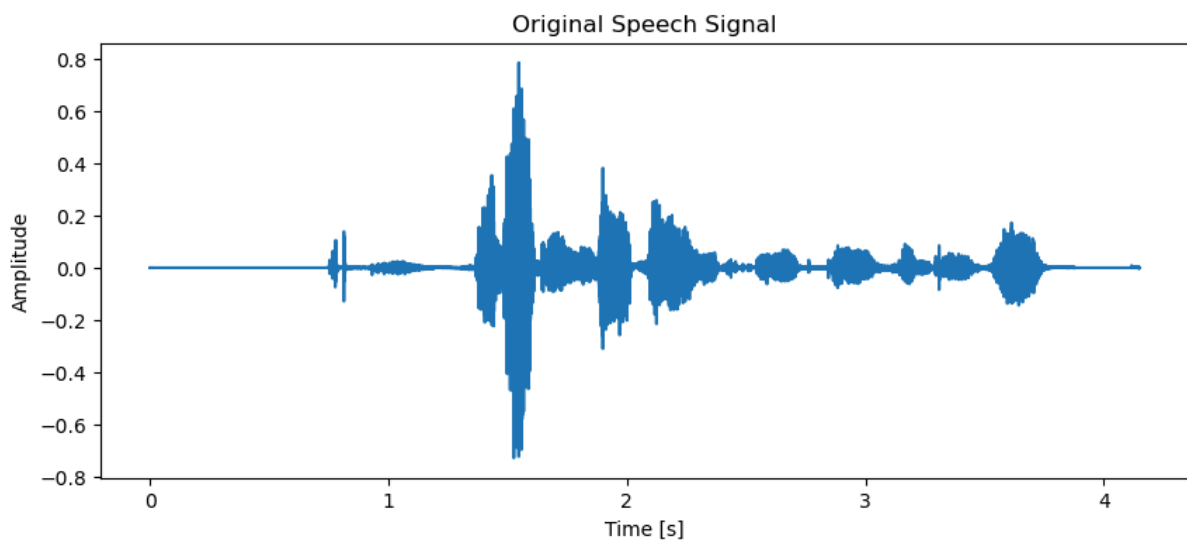
```
#preprocess the signal
#converting stereo to mono if required
if speech_signal.ndim > 1:
```

```
speech_signal = speech_signal.mean(axis=1)

#normalise amplitude
speech_signal = speech_signal / np.max(np.abs(speech_signal))

#pre-emphasis to enhance higher frequencies
speech_signal = signal.lfilter([1, -0.97], [1], speech_signal)

#plotting waveform
plt.figure(figsize=(10,4))
plt.plot(np.linspace(0, len(speech_signal)/fs, num=len(speech_signal)),
speech_signal)
plt.title('Original Speech Signal')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.show()
```



```
#lpc computation
frame_length = 2048
speech_frame = speech_signal[:frame_length] * np.hamming(frame_length)
order = 12 #LPC order

lpc_coeffs = librosa.lpc(speech_frame, order=order)
print("LPC Coefficients")
print(lpc_coeffs)
```

LPC Coefficients

```
[1.          0.95211518 0.81392142 0.70177554 0.62158903 0.58978223  
 0.50559597 0.41958294 0.34884044 0.26719937 0.19236896 0.12692688  
 0.04697909]
```

```
#reconstructing the original signal using LPC
```

```
residual = signal.lfilter(lpc_coeffs, [1], speech_frame)
```

```
reconstructed_signal = signal.lfilter([1], lpc_coeffs, residual)
```

```
plt.figure(figsize=(12,6))
```

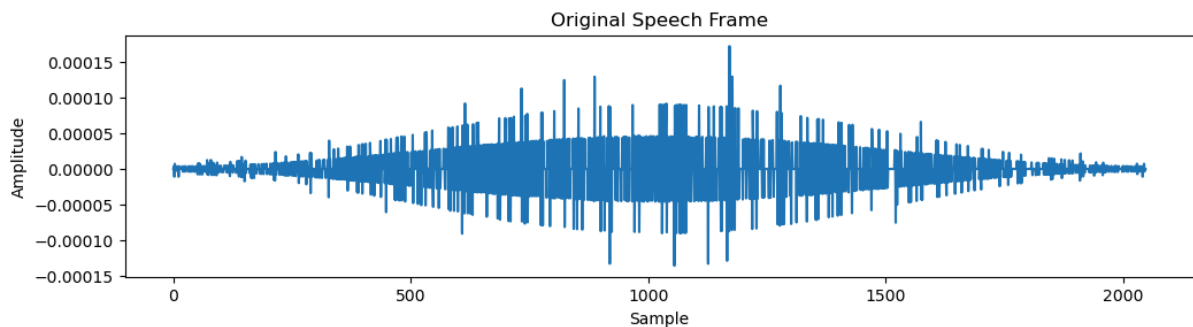
```
plt.subplot(2, 1, 1)
```

```
plt.plot(speech_frame)
```

```
plt.title('Original Speech Frame')
```

```
plt.xlabel('Sample')
```

```
plt.ylabel('Amplitude')
```



```
plt.subplot(2,1,2)
```

```
plt.plot(reconstructed_signal)
```

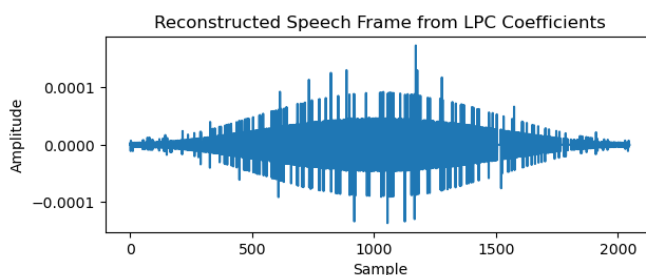
```
plt.title('Reconstructed Speech Frame from LPC Coefficients')
```

```
plt.xlabel('Sample')
```

```
plt.ylabel('Amplitude')
```

```
plt.tight_layout()
```

```
plt.show()
```



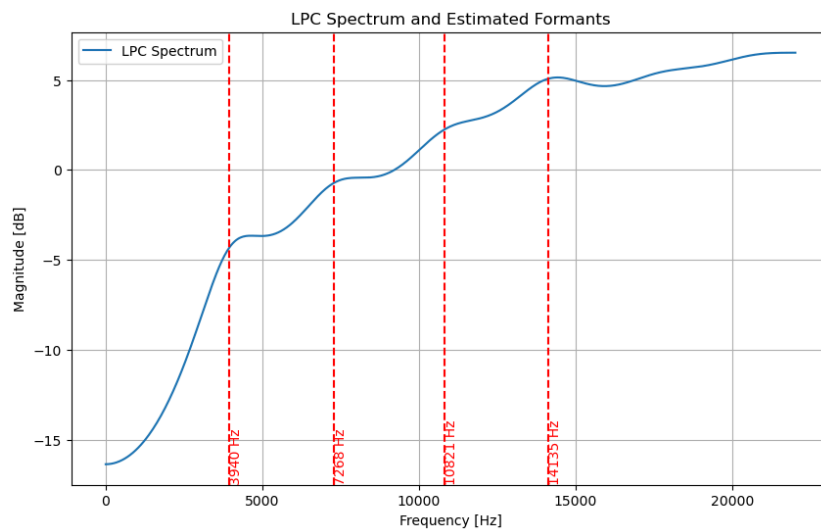
```
#estimating the formants (resonant frequencies)
rts = np.roots(lpc_coeffs)
rts = [r for r in rts if np.abs(r) < 1] #keeping stable poles
angz = np.angle(rts)
frequencies = angz * (fs / (2 * np.pi))
frequencies = frequencies[frequencies>90] #remove low freqs
frequencies = np.sort(frequencies)
formants = frequencies[:4]

print("\nEstimated Formant Frequencies(Hz): ")
for idx, freq in enumerate(formants):
    print(f"Formant F{idx+1}: {freq:2f} Hz")

Estimated Formant Frequencies(Hz):
Formant F1: 3940.487719 Hz
Formant F2: 7268.128681 Hz
Formant F3: 10820.694130 Hz
Formant F4: 14134.860274 Hz

#plotting LPC spectrum and formants
w, h = signal.freqz(1, lpc_coeffs, worN=512, fs=fs)
plt.figure(figsize=(10,6))
plt.plot(w, 20 * np.log10(np.abs(h)), label='LPC Spectrum')
plt.title("LPC Spectrum and Estimated Formants")
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')
plt.grid(True)
for freq in formants:
    plt.axvline(x=freq, color='red', linestyle='--')
    plt.text(freq, plt.ylim()[0], f'{freq:.0f} Hz', rotation=90,
color='red', verticalalignment='bottom')

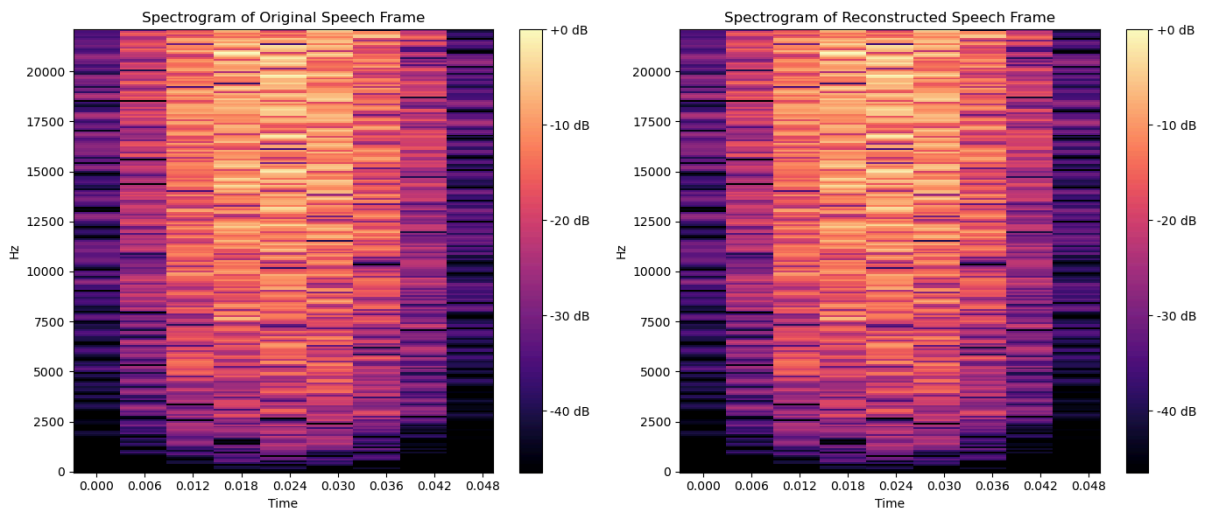
plt.legend()
plt.show()
```



```
#spectrogram comparison
n_fft = 512
hop_length = 256
S_original = np.abs(librosa.stft(speech_frame, n_fft=n_fft,
hop_length=hop_length))
S_reconstructed = np.abs(librosa.stft(reconstructed_signal,
n_fft=n_fft, hop_length=hop_length))

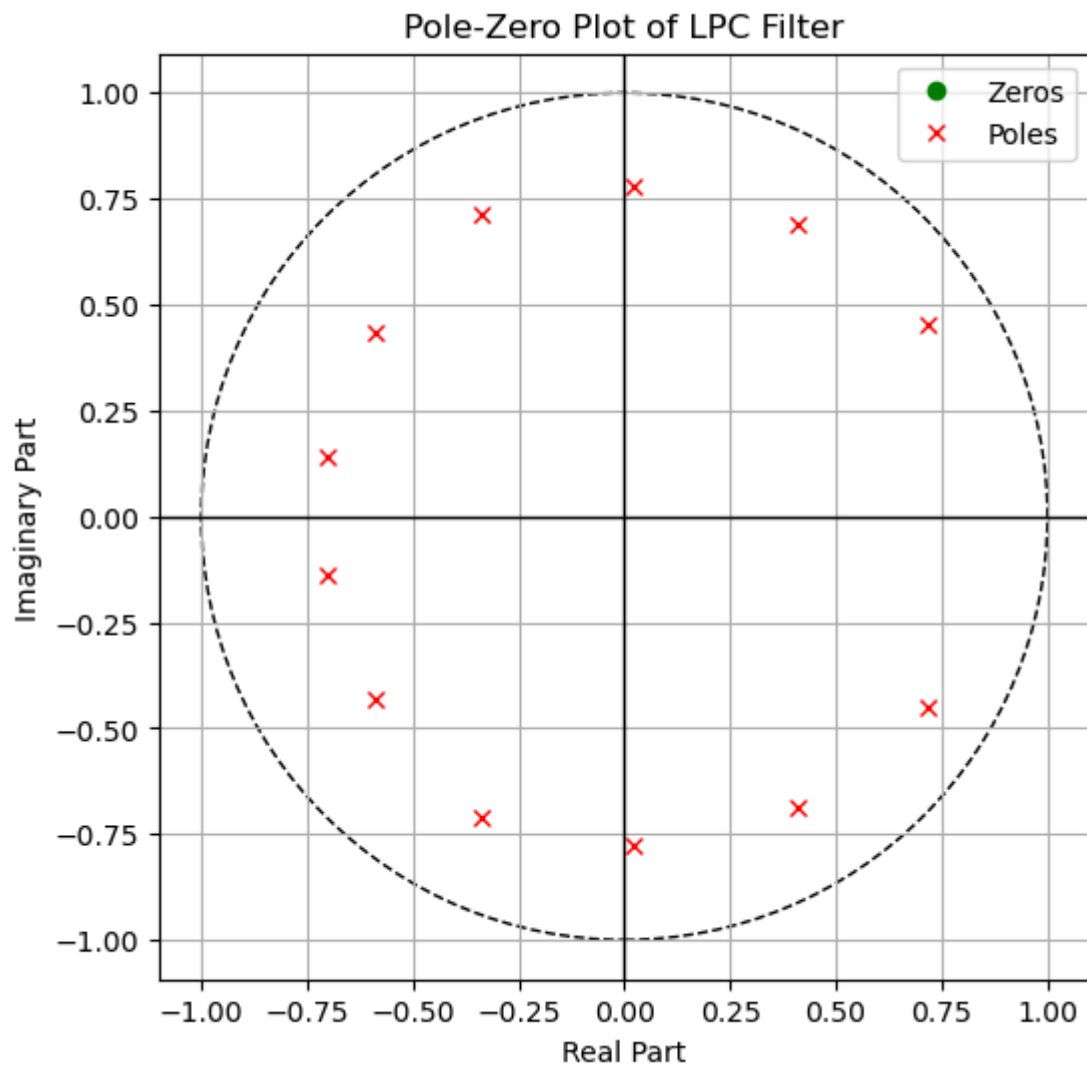
plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
librosa.display.specshow(librosa.amplitude_to_db(S_original,
ref=np.max),
                        sr=fs, hop_length=hop_length, x_axis='time',
y_axis='linear')
plt.title('Spectrogram of Original Speech Frame')
plt.colorbar(format='%+2.0f dB')

plt.subplot(1, 2, 2)
librosa.display.specshow(librosa.amplitude_to_db(S_reconstructed,
ref=np.max),
                        sr=fs, hop_length=hop_length, x_axis='time',
y_axis='linear')
plt.title('Spectrogram of Reconstructed Speech Frame')
plt.colorbar(format='%+2.0f dB')
plt.tight_layout()
plt.show()
```



```
#pole-zero plot
def zplane(b, a, title='Pole-Zero Plot'):
    plt.figure(figsize=(6, 6))
    zeros = np.roots(b)
    poles = np.roots(a)
    uc = patches.Circle((0,0), radius=1, fill=False, color='black',
ls='dashed')
    plt.gca().add_patch(uc)
    plt.axhline(0, color='black', lw=1)
    plt.axvline(0, color='black', lw=1)
    plt.plot(np.real(zeros), np.imag(zeros), 'go', label='Zeros')
    plt.plot(np.real(poles), np.imag(poles), 'rx', label='Poles')
    plt.title(title)
    plt.xlabel('Real Part')
    plt.ylabel('Imaginary Part')
    plt.legend()
    plt.grid(True)
    plt.axis('equal')
    plt.show()

zplane([1], lpc_coeffs, title='Pole-Zero Plot of LPC Filter')
```



Inference

LPC helps in achieving efficient data compression and parameter estimation.

Lab Exercise V – Time Alignment and Normalization

Aim:

To align two speech sequences of the same word spoken at different speeds using **Linear Time Normalization (LTN)** and analyze how time alignment helps in matching temporal patterns.

Given Data:

- **Signal 1 (Reference):** [0.2, 0.4, 0.6, 0.8, 1.0, 0.8, 0.6, 0.4, 0.2]
- **Signal 2 (Test):** [0.2, 0.3, 0.5, 0.7, 0.9, 1.0, 0.9, 0.7, 0.5, 0.4, 0.3, 0.2]

Tasks / Questions:

1. Plot both speech signals to observe their differences in length and amplitude patterns.
2. Perform Linear Time Normalization on Signal 2 to match the length of Signal 1.
3. Compute the alignment between Signal 1 and the normalized Signal 2.
4. Plot the alignment path, showing how each sample in Signal 1 corresponds to a sample in Signal 2.
5. Write an inference on how Linear Time Normalization aligns the two speech signals.

Expected Output:

- Two initial plots showing that Signal 2 is longer (slower speech) than Signal 1.
- A normalized version of Signal 2 that has the same number of samples as Signal 1 (resampled or interpolated).
- An alignment plot showing near-linear correspondence between both signals after normalization.

Code with Results

#(a) to plot both speech signals to observe their differences in length and amplitude patterns

#importing dependencies

import numpy as np

import matplotlib.pyplot as plt

from scipy.interpolate import interp1d

#defining the signals

```
signal1 = np.array([0.2, 0.4, 0.6, 0.8, 1.0, 0.8, 0.6, 0.4, 0.2])
```

```
signal2 = np.array([0.2, 0.3, 0.5, 0.7, 0.9, 1.0, 0.9, 0.7, 0.5, 0.4, 0.3, 0.2])
```

#plotting both signals to observe difference

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1,2,1)
```

```
plt.plot(signal1, marker='o')
```

```
plt.title('signal 1 (reference)')
```

```
plt.xlabel('sample index')
```

```
plt.ylabel('amplitude')
```

```
plt.grid(True)
```

```
plt.subplot(1,2,2)
```

```
plt.plot(signal2, marker='o')
```

```
plt.title('signal 2 (test)')
```

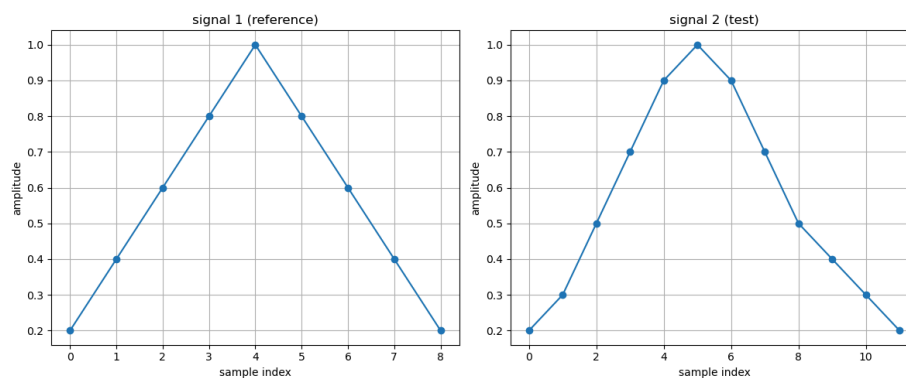
```
plt.xlabel('sample index')
```

```
plt.ylabel('amplitude')
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```



#(b) performing linear time normalisation for the two sequences

```
len1 = len(signal1)
len2 = len(signal2)
x_old = np.linspace(0, 1, len2)
f = interp1d(x_old, signal2, kind='linear')
x_new = np.linspace(0, 1, len1)
signal2_normalized = f(x_new)
```

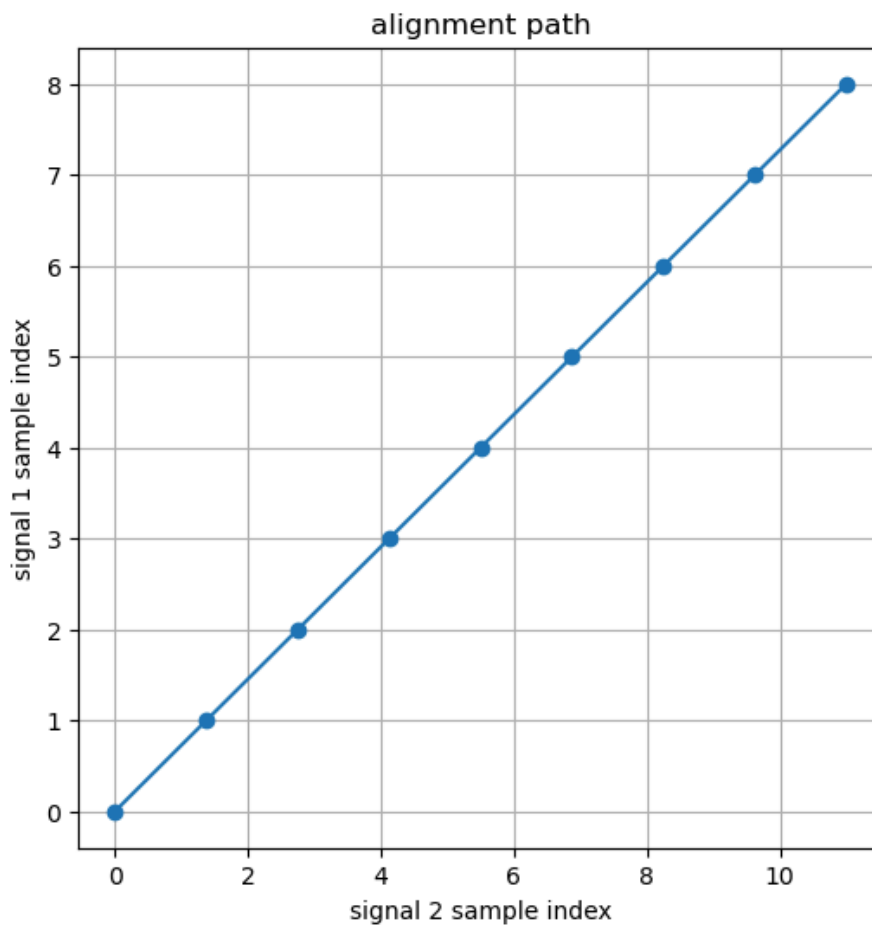
#(c) computing the alignment between signal 1 and the normalised signal 2

```
alignment = signal1 - signal2_normalized
mse = np.mean(alignment ** 2)
print(f"mean squared error after linear time normalisation: {mse:.4f}")
```

→ mean squared error after linear time normalisation: 0.0048

#(d) plotting the alignment path, showing how each sample in signal 1 corresponds to a sample in signal 2

```
signal1_indices = np.arange(len1)
signal2_indices = np.linspace(0, len2 - 1, len1)
plt.figure(figsize=(6,6))
plt.plot(signal2_indices, signal1_indices, marker='o')
plt.title('alignment path')
plt.xlabel('signal 2 sample index')
plt.ylabel('signal 1 sample index')
plt.grid(True)
plt.show()
```



Inference:

Linear Time Normalization aligns the 2 signals by scaling the axis on signal 2 uniformly so that it matches signal 1. The method assumes that the speed difference between the signals is consistent throughout the entire duration. LTN may not work for non-linear variations.

Lab Exercise VI – Dynamic Time Warping (DTW)

Aim:

To compare and align two numerical sequences using **Dynamic Time Warping (DTW)** and evaluate their similarity based on the DTW distance.

Given Data:

- **Vector 1:** [2, 3, 4, 6, 8, 7, 6, 5, 4, 3, 2]
- **Vector 2:** [2, 4, 6, 7, 7, 6, 5, 5, 4, 3, 2, 2, 1]

Tasks / Questions:

1. Plot both vectors to visualize their patterns.
2. Implement the Dynamic Time Warping algorithm.
3. Compute the accumulated cost matrix.
4. Find and visualize the optimal warping path.
5. Calculate the DTW distance between the vectors.
6. Write an inference explaining how the warping path aligns the two vectors and what the DTW distance reveals about their similarity.

Expected Output:

- Two plots showing that Vector 2 is stretched and slightly shifted relative to Vector 1.
- A computed cost matrix illustrating cumulative alignment costs between samples.
- A plotted **warping path** showing non-linear mapping between the two sequences (allowing compression and stretching along the time axis).
- A **DTW distance value**, indicating how similar the two vectors are (a smaller DTW distance means greater similarity).

Code with Results:

```
 #(a) plotting both vectors to visualise their patterns
```

```
 #importing dependencies
```

```
 import numpy as np
```

```
 import matplotlib.pyplot as plt
```

#defining the vectors

```
vector1 = np.array([2, 3, 4, 6, 8, 7, 6, 5, 4, 3, 2])
```

```
vector2 = np.array([2, 4, 6, 7, 7, 6, 5, 5, 4, 3, 2, 1])
```

#plotting both signals to visualise their patterns

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(vector1, marker='o', label='vector 1')
```

```
plt.title('vector 1')
```

```
plt.xlabel('index')
```

```
plt.ylabel('value')
```

```
plt.grid(True)
```

```
plt.legend()
```

```
plt.subplot(1, 2, 2)
```

```
plt.plot(vector2, marker='o', color='orange', label='vector 2')
```

```
plt.title('vector 2')
```

```
plt.xlabel('index')
```

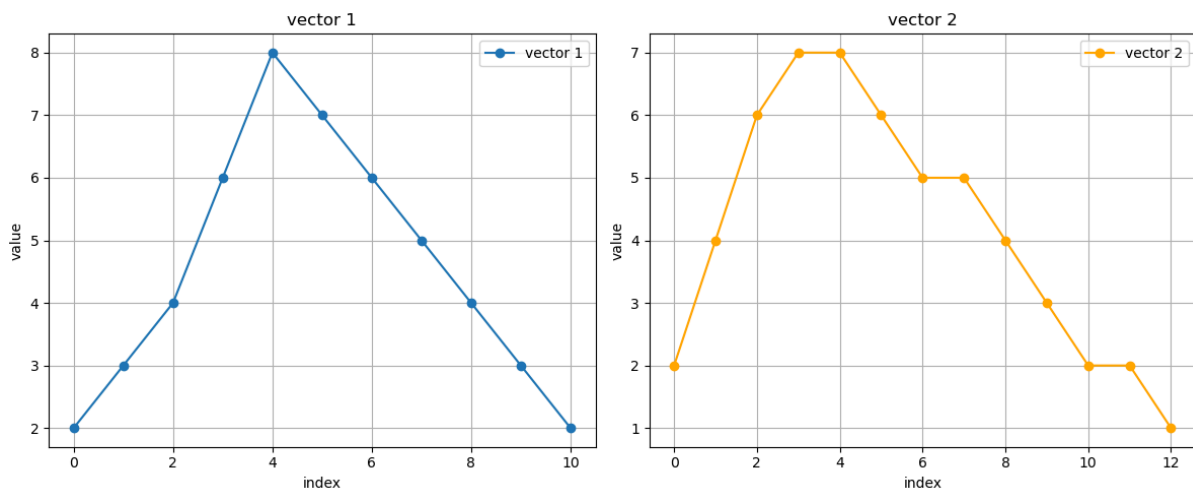
```
plt.ylabel('value')
```

```
plt.grid(True)
```

```
plt.legend()
```

```
plt.tight_layout()
```

```
plt.show()
```



#(b) implementing dynamic time warping (DTW) algorithm

```
def dtw(vector1, vector2):
```

```
    n = len(vector1)
```

```
    m = len(vector2)
```

```
    dtw_matrix = np.full((n+1, m+1), np.inf)
```

```
    dtw_matrix[0,0] = 0
```

```
    for i in range(1, n+1):
```

```
        for j in range(1, m+1):
```

```
            cost = abs(vector1[i-1] - vector2[j-1])
```

```
            dtw_matrix[i, j] = cost + min(
```

```
                dtw_matrix[i-1, j],
```

```
                dtw_matrix[i, j-1],
```

```
                dtw_matrix[i-1, j-1]
```

```
            )
```

```
    return dtw_matrix
```

#(c) computing the accumulated cost matrix using the DTW algorithm

```
dtw_matrix = dtw(vector1, vector2)
```

#visualising the accumulated cost matrix

```
plt.figure(figsize=(8,6))
```

```
plt.imshow(dtw_matrix[1:, 1:], interpolation='nearest', cmap='Blues')
```

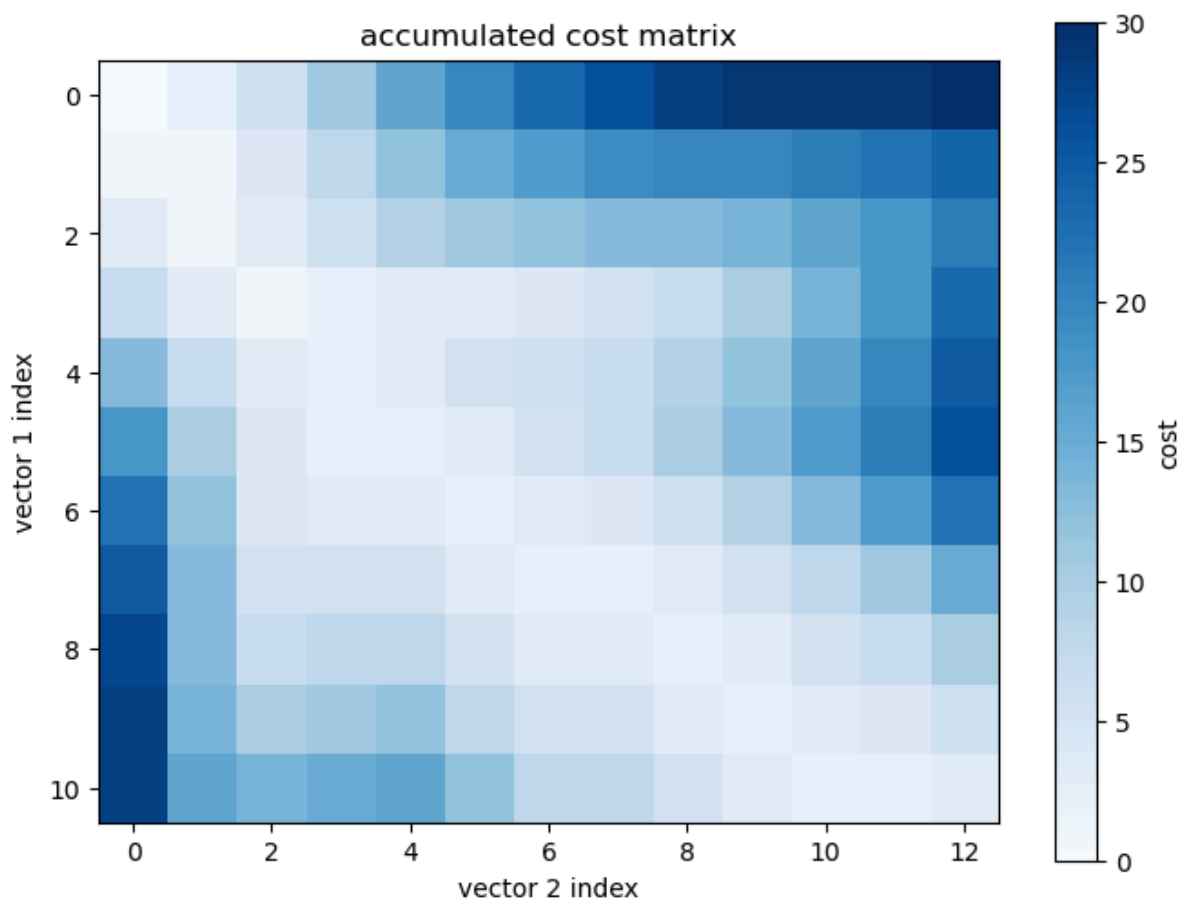
```
plt.title('accumulated cost matrix')
```

```
plt.xlabel('vector 2 index')
```

```
plt.ylabel('vector 1 index')
```

```
plt.colorbar(label='cost')
```

```
plt.show()
```



#(d) finding and visualising the warping path

```
def dtw_path(dtw_matrix):
```

```
i, j = np.array(dtw_matrix.shape) - 1

path = [(i-1, j-1)]

while (i > 1) or (j > 1):

    directions = [(i-1, j-1), (i-1, j), (i, j-1)]

    costs = [dtw_matrix[d] for d in directions]

    min_cost_index = np.argmin(costs)

    i, j = directions[min_cost_index]

    path.append((i-1, j-1))

path.reverse()

return path

#computing the warping path

path = dtw_path(dtw_matrix)

#visualising the warping path on the accumulated cost matrix

path_x, path_y = zip(*path)

plt.figure(figsize=(8, 6))

plt.imshow(dtw_matrix[1:,1:], interpolation='nearest', cmap='Blues')

plt.plot(path_y, path_x, '-o', color='red')

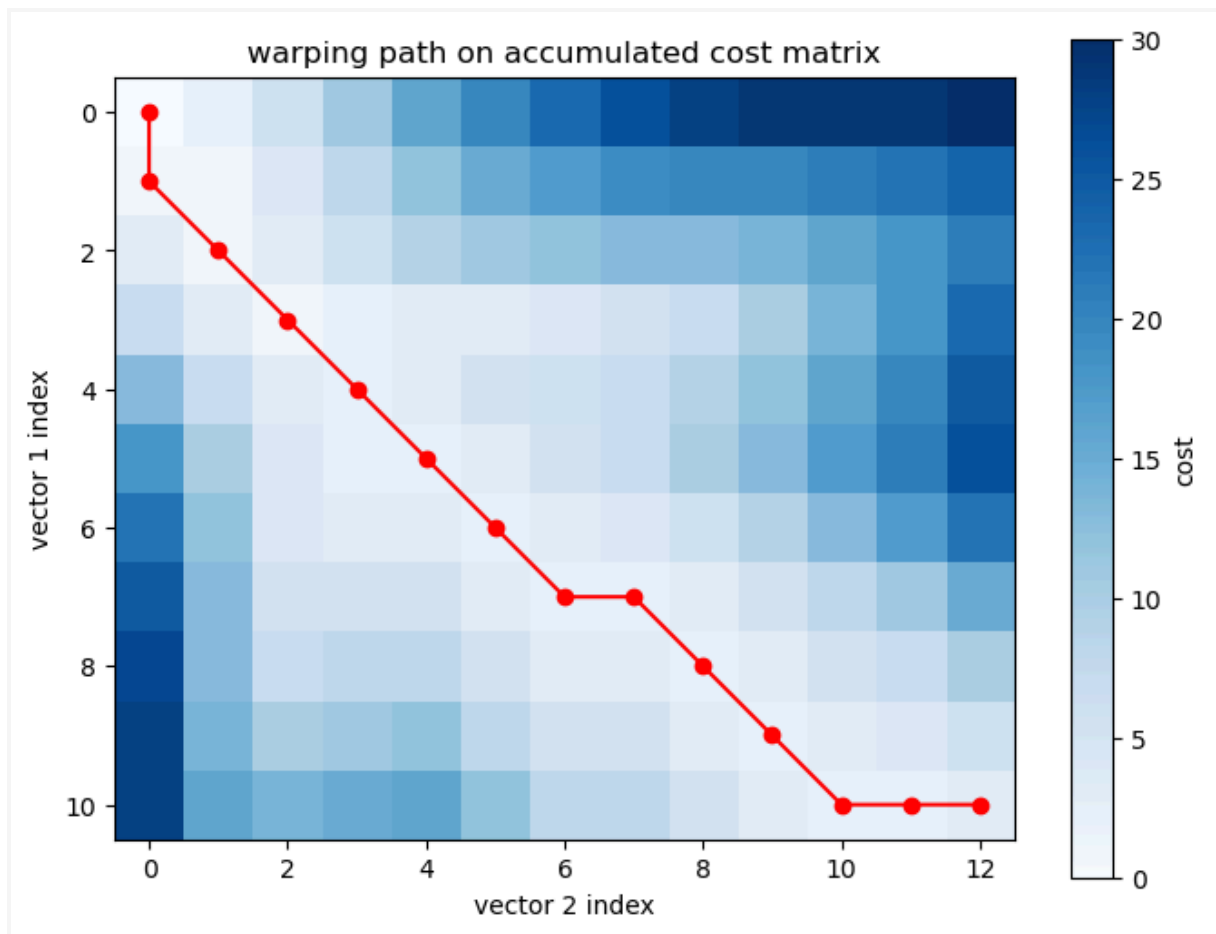
plt.title('warping path on accumulated cost matrix')

plt.xlabel('vector 2 index')

plt.ylabel('vector 1 index')

plt.colorbar(label='cost')

plt.show()
```

```
#calculating DTW distance between the vectors
```

```
dtw_distance = dtw_matrix[-1, -1]
```

```
print(f"dtw distance between vector 1 and vector 2: {dtw_distance:.2f}")
```

→ dtw distance between vector 1 and vector 2: 3.00

Inference

- The warping path tells us how each element in vector 1 has been aligned with vector 2 elements given by the DTW algorithm. The path is not strictly diagonal, as some elements are aligned with multiple elements in the vector.
- The DTW distance of 3.00 is the minimal cost required to align the two vectors, if its a lower DTW distance, then it suggests higher similarity, higher distance equals dissimilarity.
- We can infer from 3.00 (low DTW distance) that the vectors are quite similar.

Lab Question VII: Discrete Time Warping

You are given two short audio recordings.

Audio 1 is your own voice saying the word “hello”.

Audio 2 is your friend’s voice saying the word “hello”.

Your task is to compare these two signals using Dynamic Time Warping.

Tasks for the lab

1. Record your own voice saying “hello” and store it as Signal 1.
2. Record your friend saying “hello” and store it as Signal 2.
3. Convert both audio files into numerical time series by extracting their waveform data.
4. Normalize both signals so they have comparable amplitude ranges.
5. Apply Dynamic Time Warping on the two signals.
6. Produce the alignment path and compute the total DTW distance.
7. Interpret the results. Explain whether the signals are similar, and describe how DTW helps match two audio patterns that do not align perfectly in time.

Code with Results:

#(a) plotting both vectors to visualise their patterns

#importing dependencies

import numpy as np

import matplotlib.pyplot as plt

#defining the vectors

vector1 = np.array([2, 3, 4, 6, 8, 7, 6, 5, 4, 3, 2])

vector2 = np.array([2, 4, 6, 7, 7, 6, 5, 5, 4, 3, 2, 2, 1])

#loading .wav files and extracting MFCC features

import librosa

def load_wav_mfcc(file_path):

"""Load .wav file and extract MFCC features"""

y, sr = librosa.load(file_path)

mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)

Normalize to reasonable scale

mfcc = (mfcc - np.mean(mfcc)) / np.std(mfcc)

return mfcc.T # Return as time x features

#load the .wav files

try:

audio1_features = load_wav_mfcc('signal1_hello.wav')

audio2_features = load_wav_mfcc('signal2_hello.wav')

print(f'Audio 1 shape: {audio1_features.shape}')

print(f'Audio 2 shape: {audio2_features.shape}')

#use audio features instead of vectors for DTW

vector1 = audio1_features

vector2 = audio2_features

print("Using audio MFCC features for DTW analysis")

except FileNotFoundError:

print("Audio files not found, using original vectors")

except ImportError:

```
    print("librosa not installed, using original vectors")
```

```
Audio 1 shape: (87, 13)
```

```
Audio 2 shape: (87, 13)
```

```
Using audio MFCC features for DTW analysis
```

```
#plotting both signals to visualise their patterns
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(vector1, marker='o', label='vector 1')
```

```
plt.title('vector 1')
```

```
plt.xlabel('index')
```

```
plt.ylabel('value')
```

```
plt.grid(True)
```

```
plt.legend()
```

```
plt.subplot(1, 2, 2)
```

```
plt.plot(vector2, marker='o', color='orange', label='vector 2')
```

```
plt.title('vector 2')
```

```
plt.xlabel('index')
```

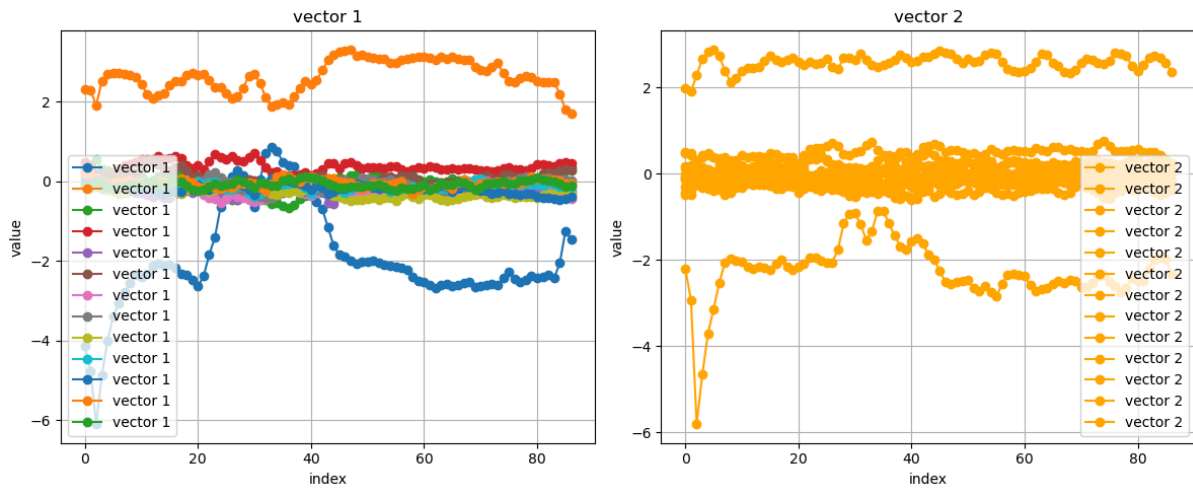
```
plt.ylabel('value')
```

```
plt.grid(True)
```

```
plt.legend()
```

```
plt.tight_layout()
```

plt.show()



#(b) implementing dynamic time warping (DTW) algorithm

```
def dtw(vector1, vector2):

    n = len(vector1)

    m = len(vector2)

    dtw_matrix = np.full((n+1, m+1), np.inf)

    dtw_matrix[0,0] = 0

    for i in range(1, n+1):

        for j in range(1, m+1):

            #handle both scalar and vector features

            if vector1.ndim == 1: #scalar values

                cost = abs(vector1[i-1] - vector2[j-1])

            else: #MFCC features (vectors)

                cost = np.linalg.norm(vector1[i-1] - vector2[j-1])

            dtw_matrix[i, j] = cost + min(

                dtw_matrix[i-1, j],

                dtw_matrix[i, j-1],
```

```
        dtw_matrix[i-1, j-1]

    )

    return dtw_matrix

#(c) computing the accumulated cost matrix using the DTW algorithm

dtw_matrix = dtw(vector1, vector2)

#visualising the accumulated cost matrix

plt.figure(figsize=(8,6))

plt.imshow(dtw_matrix[1:, 1:], interpolation='nearest', cmap='Blues')

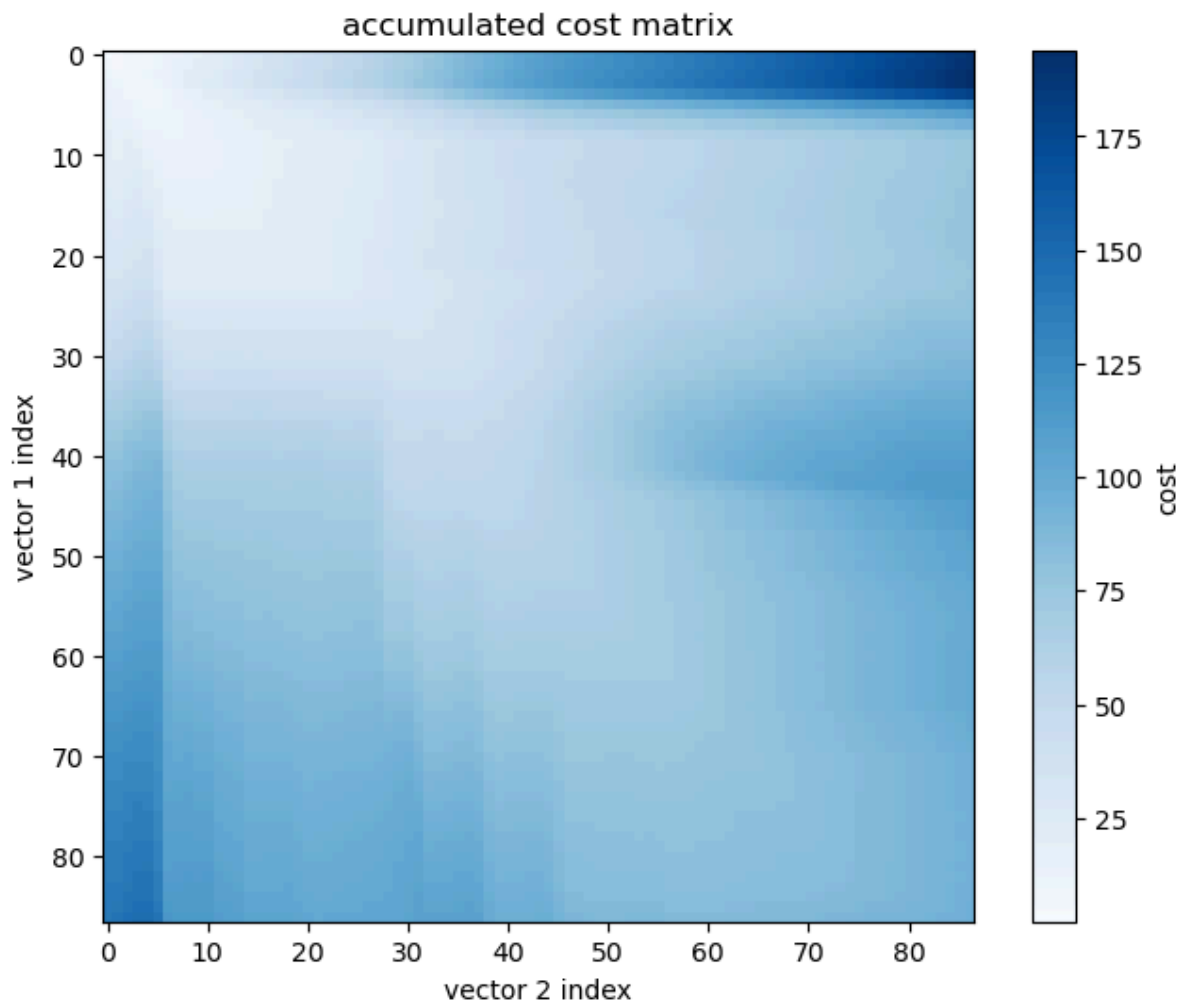
plt.title('accumulated cost matrix')

plt.xlabel('vector 2 index')

plt.ylabel('vector 1 index')

plt.colorbar(label='cost')

plt.show()
```



#(d) finding and visualising the warping path

```
def dtw_path(dtw_matrix):
```

```
    i, j = np.array(dtw_matrix.shape) - 1
```

```
    path = [(i-1, j-1)]
```

```
    while (i > 1) or (j > 1):
```

```
        directions = [(i-1, j-1), (i-1, j), (i, j-1)]
```

```
        costs = [dtw_matrix[d] for d in directions]
```

```
        min_cost_index = np.argmin(costs)
```

```
        i, j = directions[min_cost_index]
```

```
        path.append((i-1, j-1))
```

```
path.reverse()

return path

#computing the warping path

path = dtw_path(dtw_matrix)

#visualising the warping path on the accumulated cost matrix

path_x, path_y = zip(*path)

plt.figure(figsize=(8, 6))

plt.imshow(dtw_matrix[1:,1:], interpolation='nearest', cmap='Blues')

plt.plot(path_y, path_x, '-o', color='red')

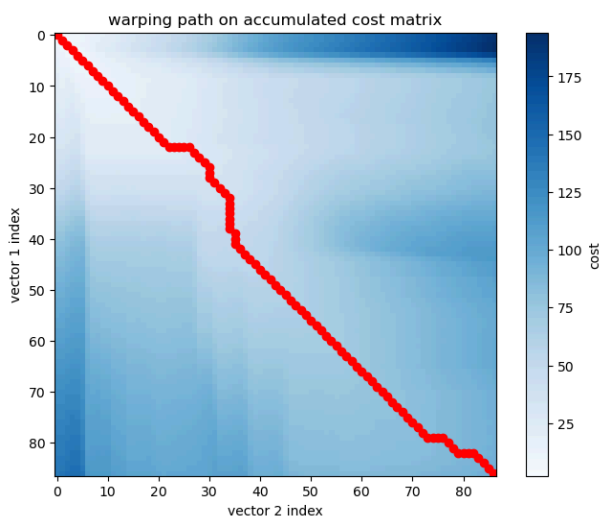
plt.title('warping path on accumulated cost matrix')

plt.xlabel('vector 2 index')

plt.ylabel('vector 1 index')

plt.colorbar(label='cost')

plt.show()
```




```
#calculating DTW distance between the vectors
```

```
dtw_distance = dtw_matrix[-1, -1]
```

```
print(f"dtw distance between vector 1 and vector 2: {dtw_distance:.2f}")
```

```
dtw distance between vector 1 and vector 2: 95.09
```

Inference

- The warped path gives us how each element in vector 1 aligns with each element in vector 2 which is given by the DTW algorithm. When the path appears, to not be strictly diagonal it simply tells us that there are elements that are aligned with multiple elements in the vector.
- The DTW distance of 95.09 is the minimal cost required to align the two vectors, if it's a lower DTW distance, then it suggests higher similarity, higher distance equals dissimilarity.
- We can infer from 95.09 (high DTW distance) that the vectors are not similar.

Lab Exercise 8 – Hidden Markov model (HMM)

Aim:

The aim of this lab experiment is to implement a Hidden Markov Model (HMM) to simulate phoneme transitions for the word ‘*speech*’ in speech processing.

Tasks / Questions:

The word speech can be divided into the following phonemes (hidden states):

- /s/
- /p/
- /ie:/ (long ee)
- /tʃ/ (ch sound)

Similarly, observations represent measurable acoustic properties such as energy, pitch, and duration. Let the initial probability of starting the phoneme, ‘/s/’ is 1.

Then, the Transition Probabilities (Probability of transitioning from one phoneme to another) are given in the below table:

From → To	/s/	/p/	/ie:/	/tʃ/
/s/	0.1	0.8	0.1	0.0
/p/	0.0	0.1	0.8	0.1
/ie:/	0.0	0.0	0.2	0.8
/tʃ/	0.2	0.0	0.0	0.8

Similarly, Emission Probabilities (the probability of emitting an observation given a phoneme) are given in the table below:

Phoneme → Observation Energy Pitch Duration			
/s/	0.7	0.2	0.1

/p/	0.5	0.3	0.2
/ie:/	0.3	0.5	0.2
/tS/	0.4	0.4	0.2

(1) Task:

- Represent the HMM parameters (initial probabilities, transition probabilities, and emission probabilities) using suitable data structures in Python.
- Write a function to neatly display the transition and emission matrices along with the initial probabilities.
- Write a program to generate a single sequence of phonemes and corresponding acoustic observations for the word speech based on the defined probabilities.
- Write an inference for the above HMM implementation

Code with Results:

```
import random
import pandas as pd

#hidden states (phonemes)
states = ["s", "p", "ie:", "tS"]

#observations
observations = ["Energy", "Pitch", "Duration"]

#initial probabilities (start at /s/)
pi = {
    "s": 1.0,
    "p": 0.0,
    "ie:": 0.0,
    "tS": 0.0
}

#transition probabilities for "speech"
#s -> p -> ie: -> tS
A = {
    "s": {"s": 0.0, "p": 1.0, "ie:": 0.0, "tS": 0.0},
    "p": {"s": 0.0, "p": 0.0, "ie:": 1.0, "tS": 0.0},
    "ie:": {"s": 0.0, "p": 0.0, "ie:": 0.0, "tS": 1.0},
    "tS": {"s": 0.0, "p": 0.0, "ie:": 0.0, "tS": 0.0},
}

#emission probabilities (given in question)
```

```
B = {  
    "s": {"Energy": 0.7, "Pitch": 0.2, "Duration": 0.1},  
    "p": {"Energy": 0.5, "Pitch": 0.3, "Duration": 0.2},  
    "ie": {"Energy": 0.3, "Pitch": 0.5, "Duration": 0.2},  
    "tS": {"Energy": 0.4, "Pitch": 0.4, "Duration": 0.2},  
}
```

#displaying matrices

```
def display_hmm(pi, A, B):  
    print("\n--- Initial Probabilities ---")  
    for state, prob in pi.items():  
        print(f'{state} : {prob}')  
  
    print("\n--- Transition Probability Matrix A ---")  
    df_A = pd.DataFrame(A)  
    print(df_A)  
  
    print("\n--- Emission Probability Matrix B ---")  
    df_B = pd.DataFrame(B)  
    print(df_B)
```

#sampling from a distribution

```
def sample_from_distribution(dist):  
    items = list(dist.items())  
    symbols = [i[0] for i in items]  
    probs = [i[1] for i in items]  
    return random.choices(symbols, weights=probs, k=1)[0]
```

#to generate from one sequence

```
def generate_sequence(pi, A, B):  
    phoneme_seq = []  
    obs_seq = []  
  
    #start state  
    current = sample_from_distribution(pi)  
    phoneme_seq.append(current)  
    obs_seq.append(sample_from_distribution(B[current]))
```

#continuing transitions until the last phoneme

```
while True:  
    if sum(A[current].values()) == 0:  
        break # end of sequence  
  
    next_state = sample_from_distribution(A[current])  
    phoneme_seq.append(next_state)  
    obs_seq.append(sample_from_distribution(B[next_state]))  
  
    current = next_state
```

```
return phoneme_seq, obs_seq

display_hmm(pi, A, B)

phonemes, observations_generated = generate_sequence(pi, A, B)

print("\nGenerated Phoneme Sequence:", phonemes)
print("Generated Observations:", observations_generated)

--- Initial Probabilities ---
s : 1.0
p : 0.0
ie: : 0.0
tS : 0.0

--- Transition Probability Matrix A ---
      s      p  ie:   tS
s      0.0  0.0  0.0  0.0
p      1.0  0.0  0.0  0.0
ie:     0.0  1.0  0.0  0.0
tS      0.0  0.0  1.0  0.0

--- Emission Probability Matrix B ---
      s      p  ie:   tS
Energy  0.7  0.5  0.3  0.4
Pitch   0.2  0.3  0.5  0.4
Duration 0.1  0.2  0.2  0.2

Generated Phoneme Sequence: ['s', 'p', 'ie:', 'tS']
Generated Observations: ['Energy', 'Pitch', 'Pitch', 'Energy']
```

Inference

- In this experiment, we modeled the pronunciation of the word "speech" using a Hidden Markov Model (HMM). The phonemes /s/ → /p/ → /ie:/ → /tS/ were treated as hidden states, while the acoustic features such as energy, pitch, and duration were considered as observable outputs. By defining initial, transition, and emission probabilities, we generated a sequence of phonemes and corresponding acoustic observations.
- The results show that the HMM successfully represents how speech sounds transition in a structured manner. The model ensures that the sequence of phonemes follows the natural pronunciation order of the word. Although the phoneme transitions are deterministic in this case (fixed path), the emitted observations vary due to probabilistic emission behavior. This demonstrates the core capability of HMMs to connect unobservable speech units (phonemes) with measurable acoustic signals, making them highly effective in speech recognition and language processing tasks.